

Tidal2Wiim

Codeatlas

Den Code durchstöbern. Datenwege nachvollziehen. Änderungen planen.

19 Modulsteckbriefe · 46 Klassen und Interfaces · 14 PlantUML-Modelle

Dieses Paket erklärt die vorhandene App vom Einstieg über lokale Speicherung und TIDAL-Anbindung bis zum Künstlereditor.

Die PlantUML-Modelle wurden mit KI-Unterstützung (ChatGPT) nach konkreten Vorgaben erstellt.

Die bestehenden ER-/Tabellenmodelle bleiben unverändert. Die neuen Klassen- und Sequenzmodelle ergänzen sie um die Sicht auf den Programmcode.

Wegweiser

Quellstand und Orientierung	3
Schnelle Orientierung	4
Architektur und Leseführung	6
Modulhandbuch	8
Datenaustausch und Speicherung	18
Modell 01 · Verantwortung und Datenwege	20
Modell 02 · Fachmodelle im Arbeitsspeicher	21
Modell 03 · Sitzungsverwaltung und Testbarkeit	22
Modell 04 · Künstlereditor: Eingabe, Entwurf und Speicherung	23
Modell 05 · Oberfläche: Objekte und Rückmeldungen	24
Modell 06 · Appstart: Cache und gespeicherte Sitzung	25
Modell 07 · Interaktive Anmeldung und sicheres Speichern	26
Modell 08 · Bibliothek laden: Vordergrund und Hintergrund	27
Modell 09 · Von JSON-Ressourcen zu Album-Objekten	28
Modell 10 · API 401: einmal erneuern und wiederholen	29
Modell 11 · Sortiernamen gesammelt bearbeiten und speichern	30
Modell 12 · Ein Album mehreren Kategorien zuordnen	31
Modell 13 · Coveranzeige: Datei, Download und Ersatzanzeige	32
Modell 14 · Albumdetails laden und an TIDAL übergeben	33
Fremdpakete und Tests	34
Vollständiges Klassenverzeichnis	36
Mit den Quellen weiterarbeiten	38

Die Kapitel und Modelle sind über das Inhaltsverzeichnis und die PDF-Lesezeichen erreichbar. Die 19 Module tragen feste Kennungen M01 bis M19. In jedem Steckbrief steht, welches Modell den Ablauf erklärt.

Leseseiten: A4. Modellblätter: A3. Bearbeitbare PlantUML-Quellen sowie SVG und PNG stehen im ZIP bereit.

Quellstand und Orientierung

Die Dokumentation beschreibt einen festgehaltenen Stand. Die Quellenlinks der Modulsteckbriefe führen zu genau diesem öffentlichen Commit.

Quellcode: `Code40vM/tidal2wiim-demo`

`851304ffd33e6ed9f65d74a29d146311e9a6265b`

Was bedeutet hier „Modul“?

Ein Steckbrief beschreibt eine eigene Dart-Datei unter `lib/`. Technisch bildet das Projekt ein Dart-Paket mit drei eigenen Bibliotheken. Die Ordner gruppieren Aufgaben, sind aber keine separat installierbaren Pakete.

Mit dem Künstlereditor beginnen

Ein einfacher Leseweg führt durch M05 (Entwurf), M16 (Editorseite), M07 (Speichern) und Modell 11. Hier lässt sich verfolgen, warum Eingaben zunächst im RAM bleiben und erst nach einer bewussten Speicheraktion in SQLite gelangen.

Werkzeuge und Dateien

Flutter und Dart bilden die App, SQLite hält die persönliche Ordnung lokal vor. VS Code dient als Entwicklungsumgebung; die Modellierung erfolgt durchgehend in PlantUML. GitHub hält Quellstände und Dokumentation nachvollziehbar zusammen. Die konkreten Laufzeit- und Entwicklungsabhängigkeiten stehen im Kapitel „Fremdpakete und Tests“.

Schnelle Orientierung

Die App ist ein digitales Plattenregal für TIDAL-Alben. Sie ergänzt die persönliche Ordnung nach Künstlern und Kategorien. Sie verarbeitet Metadaten und Cover; die Musik spielt anschließend die TIDAL-App ab.

Jedes Modul in einem Satz

Die Kennungen M01 bis M19 sind im Handbuch, in den Einzelsteckbriefen und in der PDF gleich. Ein Modul meint hier eine eigene Dart-Datei. Das gesamte Projekt ist ein Dart-Paket; die fachlichen Ordner sind keine einzeln installierbaren Pakete.

Modul / Datei	Aufgabe
M01 - lib/main.dart	Startet Flutter und setzt das App-Theme; öffnet HomePage.
M02 - lib/core/hilfen.dart	Bündelt Hilfsfunktionen für Text, Namen, Suche und JSON.
M03 - lib/core/sammlung.dart	Filtert und sortiert Alben; bildet Künstlerordner und den A-Z-Index.
M04 - lib/models/modelle.dart	Definiert die gemeinsamen Datenobjekte, etwa Album, Künstler und Kategorie.
M05 - lib/models/kuenstler_sortierung.dart	Hält die noch ungespeicherten Sortiernamen als bearbeitbaren Entwurf.
M06 - lib/data/lokale_datenbank.dart	Speichert und liest Metadaten, Kategorien und Künstlerpräferenzen in SQLite.
M07 - lib/data/kuenstler_sortierung_speichern.dart	Schreibt mehrere geänderte Künstler-Sortiernamen in einer Transaktion.
M08 - lib/services/tidal_auth_service.dart	Stellt die Sitzung wieder her, erneuert Tokens und kapselt den sicheren Store.
M09 - lib/services/tidal_api.dart	Liest TIDAL-JSON; behandelt Status, Cursor, Cover-URLs und Fehlermeldungen.
M10 - lib/services/cover_cache.dart	Lädt Cover und hält sie als lokale Bilddateien vor.
M11 - lib/pages/home_page.dart	Koordiniert Navigation, Login, Sammlung und den Hintergrundabgleich.
M12 - lib/pages/start_page.dart	Zeigt Startmotiv und Status; gibt Nutzeraktionen über Callbacks weiter.
M13 - lib/pages/album_detail_page.dart	Zeigt Albumdetails, lädt Titel und verwaltet Kategoriezuordnungen.
M14 - lib/pages/kuenstler_page.dart	Zeigt die bereits aufbereiteten Alben eines Künstlerordners.
M15 - lib/pages/kategorie_page.dart	Verbindet gespeicherte Kategorie-IDs mit den aktuell geladenen Alben.
M16 - lib/pages/kuenstler_sortiernamen_page.dart	Zeigt den Künstlereditor mit Suche, Eingabefeldern und Speichern.
M17 - lib/widgets/album_grid.dart	Zeigt ein wiederverwendbares Albumraster und öffnet Albumdetails.
M18 - lib/widgets/album_cover.dart	Zeigt ein Cover aus dem Dateicache oder eine Ersatzanzeige.
M19 - lib/widgets/dialoge.dart	Sammelt Dialogeingaben und gibt Ergebnisse an die aufrufende Seite zurück.

Daten folgen unterschiedlichen Wegen

HomePage koordiniert die Abläufe. Seiten erhalten Objekte und Dienste über Konstruktoren. Sie geben Nutzeraktionen oder Änderungen über Callbacks und Navigator-Ergebnisse zurück. Asynchrone Ergebnisse werden mit Future und await verarbeitet.

Daten	Weg / Ort	Warum getrennt?
Album-Metadaten	TIDAL-JSON → Albumobjekte → SQLite-Cache	Bereits geladene Alben sind beim nächsten Start schnell vorhanden.
Eigene Ordnung	Dialog/Editor → Ergebnisobjekte → SQLite	Kategorien und Sortiernamen bleiben lokal erhalten.
Coverbilder	HTTPS-Bildabruf → Dateien → AlbumCover	Bildbytes gehören zum Dateicache, nicht zu den SQL-Tabellen.
Anmeldung	Auth-Dienst ↔ sicherer Sitzungsspeicher	Tokens werden getrennt von Metadaten und Bildern gespeichert.
Wiedergabe	Album-/Titellink → externer Handler/TIDAL-App	Die App übergibt eine Inhalts-ID; sie streamt selbst keine Musik.

Drei passende Einstiege in die Modelle

1. Komponentenmodell 01: Aufgaben, Datenwege und Systemgrenzen im Überblick.
2. Klassenmodell 04: Wie Editor, Entwurf, Änderungsliste und Speicherung zusammenhängen.
3. Sequenzmodell 11: Eingeben → Änderungsliste bilden → gemeinsam speichern → Ansicht neu laden.

Für den Appstart folgt Modell 06, für den vollständigen Bibliotheksabgleich Modell 08 und für Covermodell 13. Das Handbuch verlinkt bei jedem Modul die passenden Diagramme.

Architektur und Leseführung

Der kurze Weg durch das Projekt

1. `main.dart`: Einstieg und Bibliotheksgrenzen erkennen.
2. `models/modelle.dart`: verstehen, welche Daten die App im RAM verwendet.
3. `core/sammlung.dart`: aus Alben Künstlerordner und Ansichten ableiten.
4. `pages/home_page.dart`: die Koordination von UI, Cache und Online-Abgleich verfolgen.
5. `data/` und `services/`: dauerhafte Daten, Anmeldung und externe Zugriffe unterscheiden.
6. Künstlereditor mit Diagramm 11 verfolgen: Ausgangsstand → Entwurf → bestätigte Änderungsliste → Transaktion → neu gelesene Ansicht.

Drei eigene Dart-Bibliotheken

Bibliothek	Verbindung	Bedeutung
<code>main.dart</code> plus 16 <code>part</code> -Dateien	<code>part / part of</code>	Gemeinsamer Namensraum. <code>_private</code> Namen sind innerhalb dieser Bibliothek dateiübergreifend sichtbar.
<code>models/kuenstler_sortierung.dart</code>	<code>import in main.dart</code>	Reines Bearbeitungsmodell ohne Flutter-, SQLite- oder TIDAL-Abhängigkeit.
<code>services/tidal_auth_service.dart</code>	<code>import in main.dart</code>	Gekapselte Sitzung mit öffentlichen Methoden und privatem internem Zustand.

Die fachlichen Ordner sind daher keine strengen technischen Schichten. `HomePage` ist die zentrale zustandsbehaftete Oberflächenkomponente (`StatefulWidget`) der Flutter-App. Ihr Zustand `_HomePageState` koordiniert Navigation, Ansichten und Datenbeschaffung. Die Startansicht mit Titelgrafik wird vom Widget `StartPage` aufgebaut. Datenbank und API verwenden dieselben Fachmodelle. Ein Refactoring in weitere unabhängige Bibliotheken wäre eine spätere Änderung, kein bereits vorhandener Zustand.

So funktionieren die Verbindungen

Konstruktorparameter: Beim Öffnen einer Seite werden Album, Listen, optionales Land und die bestehende `AuthService`-Referenz übergeben. Die Daten werden nicht erneut über ein Netzwerk zwischen den Dart-Dateien ausgetauscht.

Future und await: Ein `Future` bezeichnet ein späteres Ergebnis. `await` wartet innerhalb der betreffenden asynchronen Funktion. `Future.wait` startet keine eigenen Betriebssystemthreads; die begonnenen asynchronen Vorgänge können sich zeitlich überlappen. Auch die `Cover-Worker` sind asynchrone Schleifen, keine ausdrücklich erzeugten `Isolates`.

Callbacks: Eine Unterseite ruft eine übergebene Funktion auf. `onKategorienGeaendert()` sendet keinen vollständigen Datensatz. Die Elternansicht liest die betroffenen Daten anschließend erneut. Der Editor liefert seine Änderungsliste dagegen ausdrücklich an `onSpeichern(aenderungen)`.

UI-Zustand: `setState` meldet Flutter eine lokale Zustandsänderung. Es speichert nichts in SQLite. `ChangeNotifier` wird gezielt für den Sitzungsdienst verwendet; es gibt hier keinen allgemeinen Ereignisbus.

Navigator: Ein Dialog oder eine Route kann ein Ergebnis liefern. Bei Editor-Erfolg ist dies true; nur dann lädt HomePage die Einstellungen erneut. Bei Abbruch oder unverändertem Zurückgehen gibt es keinen bestätigten Speichervorgang.

Notation der PlantUML-Modelle

Durchgezogene Klassenpfeile zeigen gehaltene Referenzen, gestrichelte Pfeile eine Verwendung oder Erzeugung. Eine gestrichelte Beziehung „über IDs“ ist keine gespeicherte Objektreferenz. Realisierung und Vererbung sind nur dort eingezeichnet, wo sie im Dart-Code existieren. Die Extension zur Datenbank ist keine Unterklasse.

In Sequenzdiagrammen läuft die Zeit von oben nach unten. alt zeigt Alternativen, opt eine Bedingung, loop Wiederholung und par unabhängig gestartete asynchrone Vorgänge. Rückgabepfeile transportieren Daten oder signalisieren das Ende eines Future. Die Darstellung verdichtet den Code; sie ist kein Laufzeit-Trace.

Die Klassendiagramme zeigen zur Lesbarkeit ausgewählte Attribute und Methoden. Das vollständige Verzeichnis aller 46 Klassen/Interfaces steht separat. Hilfsfunktionen werden nicht als erfundene Dienstklassen gezeichnet.

Modulhandbuch

Je Datei: Aufgabe, Eingaben, Ergebnisse, Verbindungen und ein Hinweis. Die vollständigen Symbolnamen bleiben erhalten, damit sie direkt im Code gesucht werden können.

M01 · App starten und Dateien verbinden

`lib/main.dart`

Aufgabe: Startet Flutter mit Tidal2WiiMApp. Baut MaterialApp, dunkles Theme und HomePage auf. Verbindet 16 part-Dateien zu einer gemeinsamen Dart-Bibliothek und importiert zwei weitere eigene Bibliotheken.

Eingang: Optional: bibliothekLaden-Callback und TidalAuthService für isolierte Tests.

Ergebnis: Widget-Baum mit HomePage; selbst keine Albumdaten oder SQL-Abfragen.

Verbindungen: Bindet alle part-Dateien ein; importiert `models/kuenstler_sortierung.dart` und `services/tidal_auth_service.dart`.

Im Code: `main()`, `Tidal2WiiMApp.build()`

Hinweis: Die Ordner sind fachliche Bereiche. Technisch ist tidal2wiim ein Dart-Paket mit drei eigenen Bibliotheken.

Passende Modelle: 01, 05, 06

M02 · JSON, Suchtext und Zeitdauer aufbereiten

`lib/core/hilfen.dart`

Aufgabe: Stellt kleine Hilfsfunktionen ohne eigenen Zustand bereit. Prüft JSON-Strukturen, bereinigt Texte und wandelt **Zeitspannen im standardisierten Format nach ISO 8601** – hier beispielsweise die Spieldauer eines Musiktitels – für die Titelanzeige um.

Eingang: `Object?`, JSON-Werte, Suchtext oder `Duration?`.

Ergebnis: `Json`, `List<Json>`, `String?`, normalisierte Suchtexte sowie `Duration?` und Anzeigetexte.

Verbindungen: Wird von Modellen, API-Auswertung, Sammlung und Albumdetailseite benutzt. Kein direkter Netz- oder Datenbankzugriff.

Im Code: `Json`, `_objekt()`, `_optionalObjekt()`, `_liste()`, `_text()`, `_vergleichstext()`, `_dauerLesen()`, `_dauerAnzeige()`

Hinweis: Ein fehlender optionaler Wert darf null bleiben. Eine falsche Pflichtstruktur wird dagegen als `FormatException` gemeldet.

Datentypen: `String?` bezeichnet Text oder null. `Duration?` bezeichnet eine Zeitdauer oder null. Das ? erlaubt jeweils einen fehlenden Wert.

Passende Modelle: 09, 14

M03 · Ansichten aus vorhandenen Alben ableiten

lib/core/sammlung.dart

Aufgabe: Filtert und sortiert Alben, bildet Künstlerordner und berechnet den A-Z-Sprungindex. Definiert die Auswahlwerte für Appseite, Sammlungsansicht und Sortierung.

Eingang: List<Album>, Suchtext, Sortierung und Map<String, KuenstlerEinstellung>.

Ergebnis: Gefilterte Albumliste, List<KuenstlerOrdner> oder Map<String, int> als Buchstabenindex.

Verbindungen: HomePage ruft die Funktionen beim Aufbau der Sammlung auf. Benutzt Fachmodelle und Texthelfer.

Im Code: Sortierung, SammlungAnsicht, AppSeite, _ansichtErstellen(), _kuenstlerOrdnerErstellen(), _kuenstlerAlphabetIndex()

Hinweis: Künstlerordner entstehen im RAM. Mehrere Künstler eines Albums führen zu mehreren Ordnerzuordnungen. Die Albumsortierung verwendet TIDAL-Künstlernamen; eigene Sortiernamen steuern die Künstlerordner.

Passende Modelle: 01, 02

M04 · Gemeinsame Datenobjekte beschreiben

lib/models/modelle.dart

Aufgabe: Definiert acht Fach- und Transportmodelle für Album, Künstler, Kategorie, Einstellungen, Titel, Ordner, API-Seite und lokalen Cache. Album.ausRessourcen() setzt API-Ressourcen zu einem Album zusammen.

Eingang: Konstruktorwerte; Ressourcenindex nach type:id; SQLite-Map für KuenstlerEinstellung.

Ergebnis: Typisierte Dart-Objekte; abgeleitete Werte wie Jahr, Anzeigename und Albumanzahl.

Verbindungen: Gemeinsame Sprache von HomePage, Detailseiten, SQLite, Sammlung und Coveranzeige.

Im Code: Album, Kuenstler, Kategorie, KuenstlerEinstellung, KuenstlerOrdner, AlbumTitel, AlbumSeite, LokaleBibliothekCache

Hinweis: Kategorie hält Album-IDs. Album enthält Künstler, aber keine Titelliste. Ein final-Feld macht eine darin referenzierte Liste nicht automatisch unveränderlich.

Passende Modelle: 02, 09

M05 · Änderungen als Entwurf sammeln

`lib/models/kuenstler_sortierung.dart`

Aufgabe: Verwaltet die Ausgangszeilen und noch ungespeicherte Sortiernamen. Erkennt tatsächliche Änderungen und hält Reihenfolge sowie Suchgrundlage während des Tippens stabil.

Eingang: `Iterable<KuenstlerSortierZeile>`; Künstler-ID und Eingabetext; Suchtext.

Ergebnis: Gefilterte Zeilen, Änderungsanzahl und unveränderliche `List<KuenstlerSortierAenderung>`.

Verbindungen: Eigenständige Bibliothek ohne Imports von Flutter, SQLite oder TIDAL. Die Editor-Seite verwendet sie; eigene Unit-Tests prüfen die Logik.

Im Code: `KuenstlerSortierZeile`, `KuenstlerSortierAenderung`, `KuenstlerSortierEntwurf`; `setzen()`, `filtern()`, `aenderungen`

Hinweis: Rohtext bleibt beim Tippen erhalten. Erst der effektive Speicherwert wird getrimmt; ein leeres Feld fällt auf den ursprünglichen TIDAL-Namen zurück.

Passende Modelle: 04, 11

M06 · Strukturierte Daten lokal speichern

`lib/data/lokale_datenbank.dart`

Aufgabe: Öffnet `tidal2wiim.db` mit Schema 3 und aktiviert Fremdschlüssel. Kapselt Cache, Künstlerpräferenzen, Kategorien und Albumzuordnungen; übernimmt Migrationen und Transaktionen.

Eingang: Albumlisten mit Zeitstempel, `KuenstlerEinstellung`, Kategorienamen sowie Album- und Kategorie-IDs.

Ergebnis: `Future<LokaleBibliothekCache?>`, Einstellungs-Map, Kategorienlisten, ID-Mengen oder `Future<void>`.

Verbindungen: `HomePage`, `KategoriePage` und `AlbumDetailPage` nutzen `LokaleDatenbank.instance`. `sqlite` führt SQL aus; `path` bildet den Dateipfad.

Im Code: `LokaleDatenbank`, `albumCacheLaden()`, `albumCacheSpeichern()`, `kategorienLaden()`, `albumKategorienSetzen()`

Hinweis: Cacheaustausch und Zuordnungswechsel sind atomar. Nur `category_albums.category_id` ist ein SQL-Fremdschlüssel; Künstlerdaten liegen im Cache als JSON vor.

Passende Modelle: 01, 08, 12

M07 · Sortiernamen gemeinsam in SQLite schreiben

lib/data/kuenstler_sortierung_speichern.dart

Aufgabe: Ergänzt LokaleDatenbank um die SammelSpeicherung. Prüft IDs vorab und führt alle Änderungen in einer Transaktion aus. Verändert vorhandene eigene Anzeigenamen nicht.

Eingang: List<KuenstlerSortierAenderung> aus dem Entwurf.

Ergebnis: Future<void>; bei Fehlern Exception und Rollback statt eines halben Änderungsstands.

Verbindungen: HomePage bindet die Extension-Methode als onSpeichern-Callback an die Editor-Seite. Geschrieben wird artist_preferences.

Im Code: extension KuenstlerSortierungSpeichern on LokaleDatenbank;
kuenstlerSortiernamenSpeichern()

Hinweis: Eine Extension ist keine Unterklasse. Beim Zurücksetzen wird ein Präferenzsatz nur gelöscht, wenn kein eigener Anzeigename erhalten werden muss.

Passende Modelle: 04, 11

M08 · Anmeldung und Token-Lebenszyklus verwalten

lib/services/tidal_auth_service.dart

Aufgabe: Stellt die gemeinsame Sitzung wieder her, speichert Login-Ergebnisse und erneuert Tokens bei Bedarf. Schützt vor mehrfachen parallelen Refreshs und Ergebnissen einer inzwischen ersetzten Sitzung.

Eingang: clientId, optionale Store-/Client-/Uhr-Abhängigkeiten; TidalSession beim Login; HTTPS-URI und HTTP-Client beim Lesen.

Ergebnis: Future<String> für ein Token, Future<Response> für API-GET, Status-Getter und ChangeNotifier-Benachrichtigungen.

Verbindungen: HomePage besitzt oder erhält den Dienst und gibt dieselbe Referenz an Unterseiten weiter. SecureTidalSessionStore verwendet flutter_secure_storage.

Im Code: TidalAuthService, TidalSession, TidalSessionStore, SecureTidalSessionStore; restore(), acceptLogin(), accessToken(), get(), forget()

Hinweis: HTTP 401 führt höchstens zu einem zweiten API-GET. Temporäre Netzfehler erhalten die Sitzung. storageWarning weist auf Probleme beim Lesen, Speichern oder Entfernen der sicheren Sitzung hin.

Passende Modelle: 03, 06, 07, 10

M09 · API-Aufrufe vereinheitlichen und JSON lesen

lib/services/tidal_api.dart

Aufgabe: Baut OpenAPI-URLs, ruft den Auth-Dienst auf, prüft HTTP 200 und decodiert JSON. Liest Folgekursor, wählt eine passende Cover-URL und übersetzt Fehler in verständliche Hinweise.

Eingang: http.Client, gemeinsamer TidalAuthService, Pfadsegmente und Queryparameter; JSON-Ressourcen für Cover und Cursor.

Ergebnis: Future<Json>, String? für Cursor/Cover-URL oder ApiFehler.

Verbindungen: HomePage und AlbumDetailPage nutzen _apiLesen(). Album.ausRessourcen() nutzt _coverUrlLesen().

Im Code: _apiLesen(), _cursorLesen(), _coverUrlLesen(), _fehlerText(), ApiFehler

Hinweis: Die Datei ist überwiegend eine Funktionssammlung, keine TidalApi-Klasse. Jeder Aufruf wartet vor der Anfrage 300 ms; das ist kein vollständiger automatischer 429-Retry.

Passende Modelle: 01, 09, 10

M10 · Albumcover als Dateien zwischenspeichern

lib/services/cover_cache.dart

Aufgabe: Findet oder lädt Coverdateien im lokalen cover_cache-Verzeichnis. Bündelt laufende Downloads für denselben Zielpfad, schreibt zunächst .tmp und räumt beim Abgleich überholte Dateien auf.

Eingang: Album mit ID und Cover-URL oder List<Album> zum Vorladen; optionaler HTTP-Client.

Ergebnis: Future<File?> für ein Cover oder Future<void> nach synchronisieren().

Verbindungen: AlbumCover fragt einzelne Dateien an. HomePage startet das Vorladen nach Cacheübernahme oder vollständiger Aktualisierung.

Im Code: LokalerCoverCache.instance, dateiFuer(), synchronisieren(), _herunterladen()

Hinweis: Bis zu vier Worker gelten für synchronisieren(), nicht als globale Grenze aller Einzelabrufe. Bildbytes liegen im Dateisystem und nie als BLOB in SQLite.

Passende Modelle: 01, 06, 13

M11 · Appzustand und Abläufe koordinieren

lib/pages/home_page.dart

Aufgabe: Das zentrale StatefulWidget HomePage und sein Zustand _HomePageState verwalten Startansicht, Labor und Sammlung. Sie koordinieren Login, Cache, API-Seiten, Hintergrundabgleich, Suche, Künstlerordner, Kategorien und Navigation zum Sortiernameneditor.

Eingang: Optionale Test-Abhängigkeiten, Nutzeraktionen, Dienstmeldungen, API- und Datenbankergebnisse.

Ergebnis: Aktueller UI-Zustand; Daten und AuthService an Unterseiten; Callbacks zum Neuladen.

Verbindungen: Zentraler Auftraggeber für Datenbank, Auth, API und Covercache. Baut StartPage; öffnet Künstler-, Kategorie-, Album- und Editor-Seiten.

Im Code: HomePage, _HomePageState; _initialisieren(), _seiteLesen(), _albenLaden(), _bibliothekImHintergrundAktualisieren(), _kuenstlerSortiernamenOeffnen()

Hinweis: Die Aufteilung in Dateien trennt noch nicht sämtliche UI- und Fachlogik. Labor und Sammlung sind hier Widget-Methoden, keine eigenen Seitenklassen.

Passende Modelle: 01, 05, 06, 08, 09, 11

M12 · Startbild mit echten Bedienelementen verbinden

lib/pages/start_page.dart

Aufgabe: Zeigt die Startgrafik, dynamische Albumzahl und Statusleiste. Skaliert Bild, Texte und Trefferflächen gemeinsam und löst vier vom Elternwidget bereitgestellte Aktionen aus.

Eingang: Albumanzahl, Sitzung-/Ladeflags, Aktualisierungszeit sowie vier Callbacks.

Ergebnis: Aufrufe von onSammlungOeffnen, onLaborOeffnen, onAktualisieren und onTidalStatus.

Verbindungen: Wird von HomePage konfiguriert. _StartAktion, _StartStatusLeiste und zwei CustomPainter bilden interne UI-Bausteine.

Im Code: StartPage, _StartAktion, _StartStatusLeiste, _StartTidalZeichen, _StartBibliothekZeichen

Hinweis: StartPage liest selbst weder SQLite noch die API. Sie zeigt den gelieferten Zustand und delegiert Aktionen zurück an HomePage.

Passende Modelle: 05, 06

M13 · Album, Titel und lokale Zuordnungen anzeigen

lib/pages/album_detail_page.dart

Aufgabe: Lädt die Titelliste aus TIDAL und Kategorien aus SQLite unabhängig voneinander. Ermöglicht Kategoriezuordnung und öffnet Album-, Track- oder Videolinks extern.

Eingang: Album, gemeinsamer AuthService, optionales Land sowie Callbacks für Anmeldung und Kategorieänderungen.

Ergebnis: List<AlbumTitel> im Seitenzustand, lokale Kategorieauswahl und externe URLs; Rückmeldung nach gespeicherter Zuordnung.

Verbindungen: Aufruf durch HomePage oder AlbumGrid; nutzt _apiLesen(), LokaleDatenbank, AlbumCover und Dialoge.

Im Code: AlbumDetailPage, _AlbumDetailPageState; _titelLaden(), _kategorienBearbeiten(), _inTidalOeffnen(), _titelInTidalOeffnen()

Hinweis: Die Titelliste wird nicht dauerhaft gecacht. launchUrl() startet einen externen Handler; Play und WiiM-Auswahl erfolgen anschließend außerhalb unserer App.

Passende Modelle: 05, 12, 14

M14 · Alben eines Künstlerordners darstellen

lib/pages/kuenstler_page.dart

Aufgabe: Ist eine kleine, zustandslose Hülle: Zeigt den Anzeigenamen als Seitentitel und übergibt die zugehörigen Alben an AlbumGrid.

Eingang: KuenstlerOrdner, AuthService, optionales Land und Änderungs-/Anmelde-Callbacks.

Ergebnis: Widget-Baum aus AppBar und AlbumGrid.

Verbindungen: HomePage erstellt den Ordner zuvor mit _kuenstlerOrdnerErstellen(). AlbumGrid übernimmt die Detailnavigation.

Im Code: KuenstlerPage.build()

Hinweis: Die Seite legt keine Ordner an und fragt keine Künstlerdaten nach. Sie erhält bereits aufbereitete Objekte.

Passende Modelle: 02, 05

M15 · Gespeicherte Zuordnung mit geladenen Alben verbinden

lib/pages/kategorie_page.dart

Aufgabe: Liest die Kategorie samt Album-IDs und filtert die übergebene Albumliste. Zeigt sowohl sichtbare Alben als auch die Zahl gespeicherter Zuordnungen.

Eingang: Kategorie-ID/-Name, alle aktuell geladenen Alben, AuthService und Callbacks.

Ergebnis: Nach Jahr und Titel sortierte List<Album> für AlbumGrid; Aktualisierung nach Kategorieänderungen.

Verbindungen: Nutzt LokaleDatenbank.kategorienLaden(); AlbumGrid öffnet Details. Der Callback lädt zunächst diese Seite und danach die Elternansicht neu.

Im Code: KategoriePage, _KategoriePageState; _neuLaden()

Hinweis: Eine gespeicherte Album-ID muss nicht mehr im aktuellen Cache liegen. Deshalb können Gesamtzahl und sichtbare Anzahl voneinander abweichen.

Passende Modelle: 05, 12

M16 · Mehrere Sortiernamen bequem bearbeiten

lib/pages/kuenstler_sortiernamen_page.dart

Aufgabe: Stellt Suche, Eingabefelder, Änderungszähler und Speichern bereit. Verwaltet Controller, Fokuswechsel und Schutz vor dem Verlassen mit ungespeicherten Änderungen.

Eingang: List<KuenstlerSortierZeile> und onSpeichern-Callback.

Ergebnis: Änderungsliste an den Callback; Navigator-Ergebnis true nach erfolgreichem Speichern. Bei Abbruch kein true.

Verbindungen: HomePage liefert Zeilen und Speicherfunktion. KuenstlerSortierEntwurf hält die Fachlogik getrennt von Flutter.

Im Code: KuenstlerSortiernamenPage, _KuenstlerSortiernamenPageState; _speichern(), _verlassen(), _fokusVerschieben()

Hinweis: Bei einem Speicherfehler bleiben die Eingaben auf der Seite. Während des Speicherns sind Bearbeitung und unbeabsichtigtes Schließen gesperrt.

Passende Modelle: 04, 11

M17 · Wiederverwendbares Raster für Unterseiten

lib/widgets/album_grid.dart

Aufgabe: Zeigt Albumkarten mit Cover, Titel, Künstlern und Jahr. Öffnet beim Antippen eine AlbumDetailPage mit denselben Diensten und Callbacks.

Eingang: List<Album>, AuthService, optionales Land und zwei Callbacks.

Ergebnis: Albumraster und Navigation zur Detailseite.

Verbindungen: Wird von KuenstlerPage und KategoriePage verwendet; baut AlbumCover. Die Hauptsammlung hat derzeit ein eigenes Raster in HomePage.

Im Code: AlbumGrid.build()

Hinweis: Wiederverwendung ist hier konkret sichtbar. Es wäre ungenau zu behaupten, dass bereits jedes Raster der App dieses Widget nutzt.

Passende Modelle: 05, 14

M18 · Coverdatei laden und Bildfehler abfangen

lib/widgets/album_cover.dart

Aufgabe: Fordert ein Cover vom Dateicache an und zeigt Ladeanzeige, Datei oder Netzbild. Bei fehlender URL bzw. erfolglosem Netzbild erscheint ein Platzhalter.

Eingang: Album; beim Widget-Update werden geänderte ID oder Cover-URL erkannt.

Ergebnis: Bilddarstellung mit FutureBuilder; kein geändertes Albumobjekt.

Verbindungen: Verwendet LokalerCoverCache.dateiFuer(), Image.file und als Ersatz Image.network. Eingesetzt in Hauptsammlung, AlbumGrid und Detailseite.

Im Code: AlbumCover, _AlbumCoverState; initState(), didUpdateWidget(), _netzFallback()

Hinweis: Ein vorhandener Dateipfad garantiert noch kein decodierbares Bild. Darum besitzt auch Image.file eine Fehlerbehandlung.

Passende Modelle: 13

M19 · Eingaben sammeln und Ergebnisse zurückgeben

lib/widgets/dialoge.dart

Aufgabe: Bündelt drei zustandsbehaftete Dialoge: Kategorienname, einzelne Künstleranpassung und Album-Kategorienauswahl. Jeder Dialog besitzt seine Eingabecontroller bzw. Auswahlkopie.

Eingang: Dialogtitel/Startname, Künstlerdaten bzw. Kategorien mit ausgewählten IDs.

Ergebnis: String, `_KuenstlerDialogErgebnis` oder `Set<int>` über `Navigator.pop()`; bei Abbruch null.

Verbindungen: `HomePage` und `AlbumDetailPage` öffnen die Dialoge und speichern anschließend selbst in `SQLite`.

Im Code: `_KategorieNameDialog`, `_KuenstlerBearbeitenDialog`, `_AlbumKategorienDialog`, `_KuenstlerDialogErgebnis`; jeweilige State-Klassen

Hinweis: Die Dialoge führen keine SQL-Operationen aus. Controller werden in `dispose()` ihres eigenen Dialogzustands freigegeben.

Passende Modelle: 12

Datenaustausch und Speicherung

Datenverträge zwischen den Modulen

Verbindung	Datentyp / Übergabe	Bedeutung
API → _apiLesen()	HTTP-Response / UTF-8 JSON	HTTP 200 wird geprüft; Ergebnis ist Json = Map<String, dynamic>.
JSON → Album	Map<String, Json> nach type:id	Sammlungs-IDs werden mit Album-, Künstler- und Coverressourcen aufgelöst.
Seitenlader → Home	AlbumSeite	Enthält List<Album> und nächsten Sammlungs-Cursor, nicht die komplette Bibliothek.
SQLite → Home	LokaleBibliothekCache?	Albumliste und Speicherzeit; eine leere Tabelle ergibt null.
Home → Unterseite	Objekte und Referenzen	Album/Ordner/Listen plus dieselbe TidalAuthService-Instanz; kein kopierter Token.
Dialog → Aufrufer	String / Set<int> / Ergebnisobjekt	Navigator.pop liefert ein späteres Ergebnis; null bedeutet meist Abbruch.
Editor → Speicherfunktion	List<KuenstlerSortierAenderun g>	Nur tatsächlich geänderte Künstler; nach Erfolg pop(true) an Home.
Detailseite → Elternansicht	Future<void> Function()	onKategorienGeaendert meldet die Änderung ohne Datensatzpayload; Eltern lesen erneut.
Auth → Home	ChangeNotifier	Listener liest Status-Getter neu und ruft setState; kein Ereignisbus für die ganze App.
Covercache → Widget	Future<File?>	Bildbytes bleiben im Dateisystem. null/Dateifehler führt zum Netz-Fallback.
App → TIDAL-App	HTTPS-URL	Nur Inhalts-ID im Link; weder OAuth-Token noch Musikdatei werden übergeben.

Vier Speicherformen

Ort	Inhalt	Lebensdauer / Grenze
Arbeitsspeicher	Alben, gefilterte Ansichten, geladene Titel, Editorentwurf	Geht beim Prozessende verloren; setState aktualisiert nur die Oberfläche.
SQLite / tidal2wiim.db	Kategorien, Albumzuordnungen, Künstlerpräferenzen, Metadaten-cache	Dauerhaft im App-Bereich. Schema 3; keine Audiostreams und keine Token.
Dateien / cover_cache	Heruntergeladene Cover als .img; vorläufig .tmp	Dateiname aus bereinigter Album-ID und stabilem URL-Hash; später erneut ladbar.
Sicherer Store	Version, clientId, Access-/Refresh-Token und Ablaufzeit als JSON	Gekapselt durch TidalSessionStore; nicht in SQLite oder Coverdateien.

SQLite ist nicht das Klassenmodell

artist_preferences speichert eigene Künstlernamen und Sortiernamen. categories enthält Kategorien. category_albums bildet Kategorie/Album-Paare ab. album_cache enthält Album-Metadaten, die Reihenfolge und Künstlerdaten als JSON.

Nur category_albums.category_id → categories.id ist ein SQL-Fremdschlüssel. Auf album_cache existiert kein solcher Fremdschlüssel. Ein vollständiger Cacheaustausch löscht deshalb keine Kategoriezuordnungen. Auch artist_preferences verweist nur logisch auf TIDAL-Künstler-IDs; eine eigene Künstler-Tabelle gibt es im Ist-Schema nicht.

Ein KuenstlerOrdner entsteht aus den aktuellen Albumobjekten. AlbumSeite und LokaleBibliothekCache sind Transportobjekte. AlbumTitel wird beim Öffnen einer Detailseite geladen und nicht in der lokalen Datenbank gespeichert. Die vorhandenen ER-/Tabellenmodelle bleiben eigenständige, unveränderte Darstellungen.

Beispiel: ein neuer Sortiername

Aus Albumdaten und gespeicherten Einstellungen entstehen Editorzeilen. Die Eingabe „Wülker, Nils“ landet zunächst als Text im Entwurf. Beim Speichern entsteht ein KuenstlerSortierAenderung-Objekt mit Künstler-ID, TIDAL-Name und neuem Sortiernamen. Die Extension aktualisiert artist_preferences innerhalb einer Transaktion. Anschließend liest HomePage die Einstellungen erneut und berechnet Künstlerordner und A-Z-Index neu. Der TIDAL-Name und ein vorhandener eigener Anzeigename bleiben erhalten.

Beispiel: ein aktualisiertes Album

Die Sammlung liefert zunächst die Album-ID. Weitere API-Aufrufe liefern Album-, Künstler- und Coverressourcen. Album.ausRessourcen() erzeugt das Fachobjekt. Nach vollständigem Hintergrundabgleich wird der gesamte Metadatencache atomar ersetzt. Erst danach wird die sichtbare Albumliste ausgetauscht. Das Cover wird getrennt als Datei geladen. Ein einzelner fehlerhafter Coverdownload macht den erfolgreich gespeicherten Albumdatensatz nicht ungültig.

Systemgrenze der Wiedergabe

Album- und Titellinks enthalten eine Inhalts-ID. Sie enthalten keine Session-Tokens. Die Plattform entscheidet über den externen Handler; auf dem vorgesehenen Tablet ist dies die TIDAL-App. Dort muss die Wiedergabe gestartet werden. TIDAL Connect zum WiiM ist kein eigener Aufruf des hier untersuchten Codes.

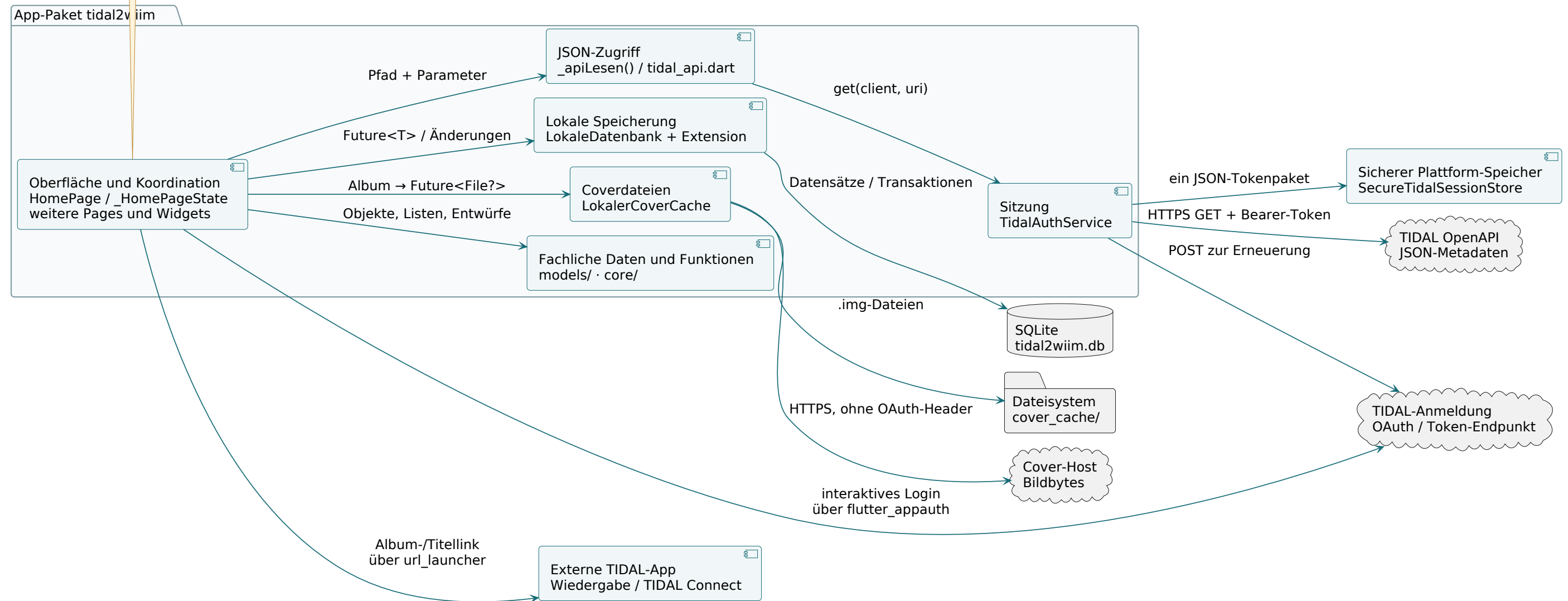
Modell 01 · Verantwortung und Datenwege

Die App verarbeitet Metadaten und persönliche Ordnung. Der Musikstream und die WiiM-Steuerung liegen außerhalb ihres Dart-Codes.

Quellstellen unter lib/, sofern nicht anders angegeben: main.dart; pages/home_page.dart; services/*; data/*

Verantwortung und Datenwege

Fachliche Gruppierung, keine starre Schichtenarchitektur.
HomePage koordiniert zugleich UI, API-Laden und lokale Daten.
callbacks und setState aktualisieren die Darstellung.



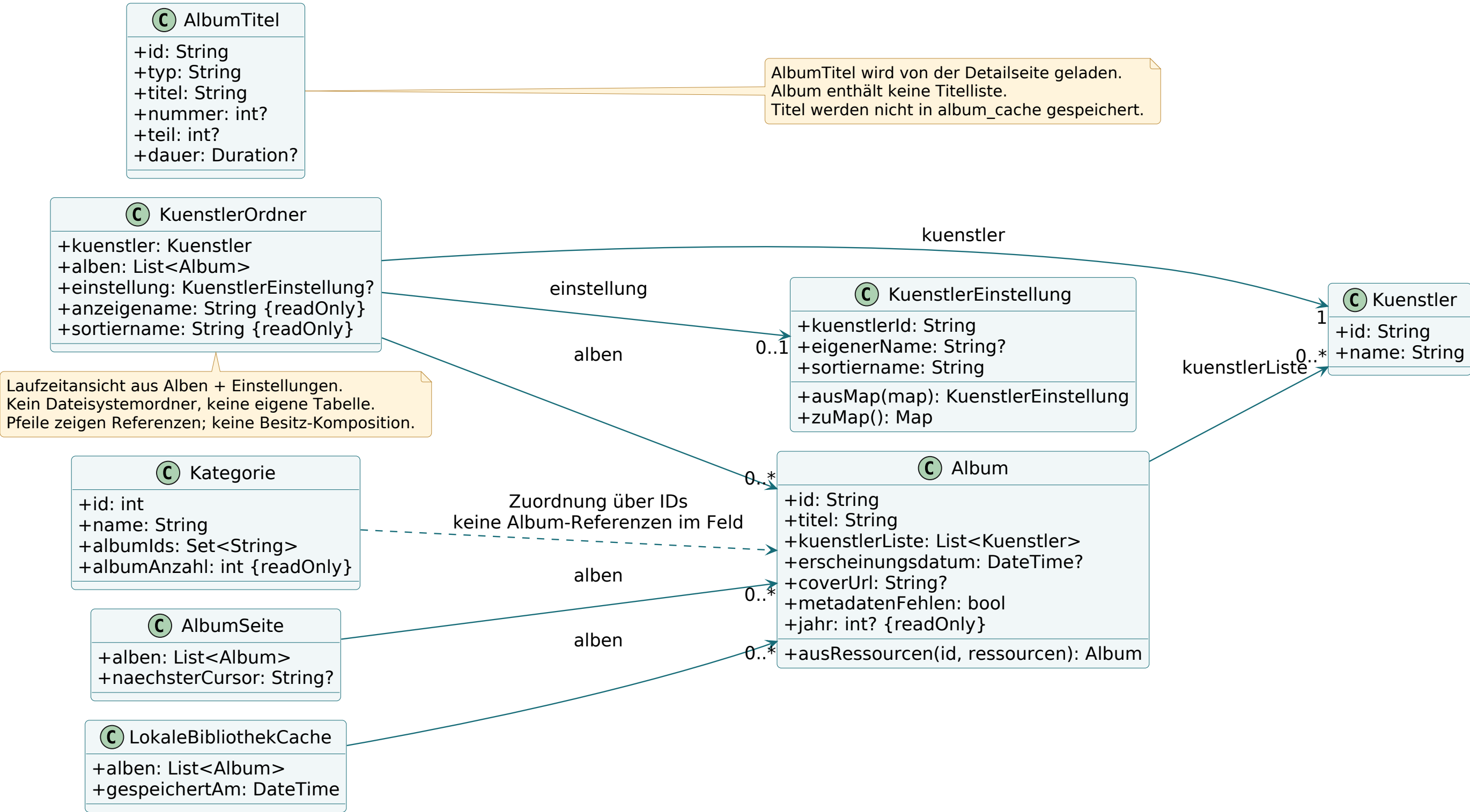
Bearbeitbare Quelle: PlantUML/01_Komponenten.puml · Vektorausgabe: SVG/01_Komponenten.svg

Modell 02 · Fachmodelle im Arbeitsspeicher

Objekte, fachliche Beziehungen und SQL-Tabellen sind unterschiedliche Sichten. Besonders wichtig: Kategorien speichern IDs, Albumtitel leben nur im Zustand der Detailseite.

Quellstellen unter lib/, sofern nicht anders angegeben: models/modelle.dart; core/sammlung.dart

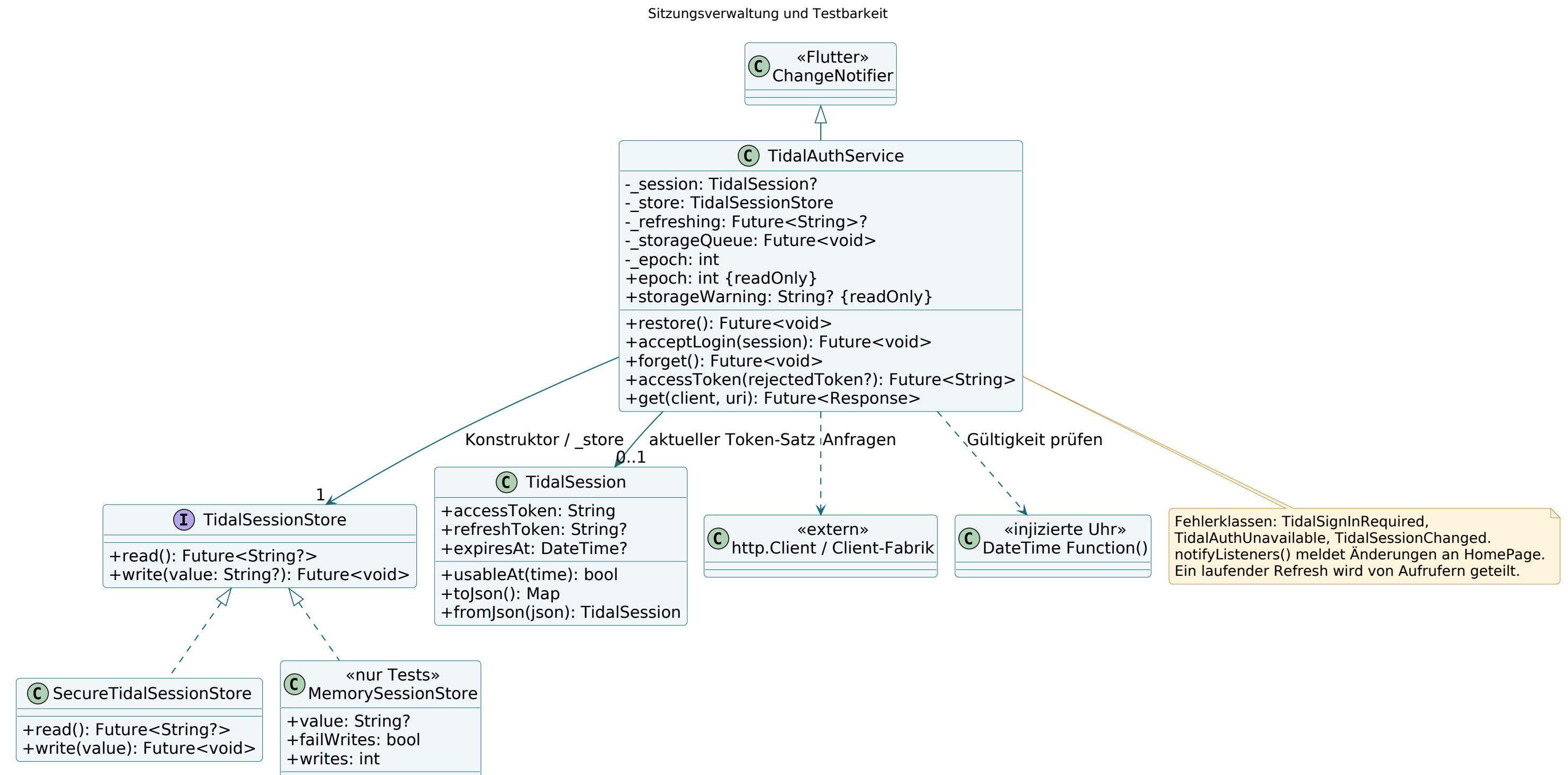
Fachmodelle im Arbeitsspeicher



Modell 03 · Sitzungsverwaltung und Testbarkeit

Store, Uhr und HTTP-Client sind austauschbar. Deshalb lassen sich Ablaufzeit, Speicherausfälle und parallele Anfragen ohne echten TIDAL-Zugang testen.

Quellstellen unter lib/, sofern nicht anders angegeben: `services/tidal_auth_service.dart`; `test/support/memory_session_store.dart`

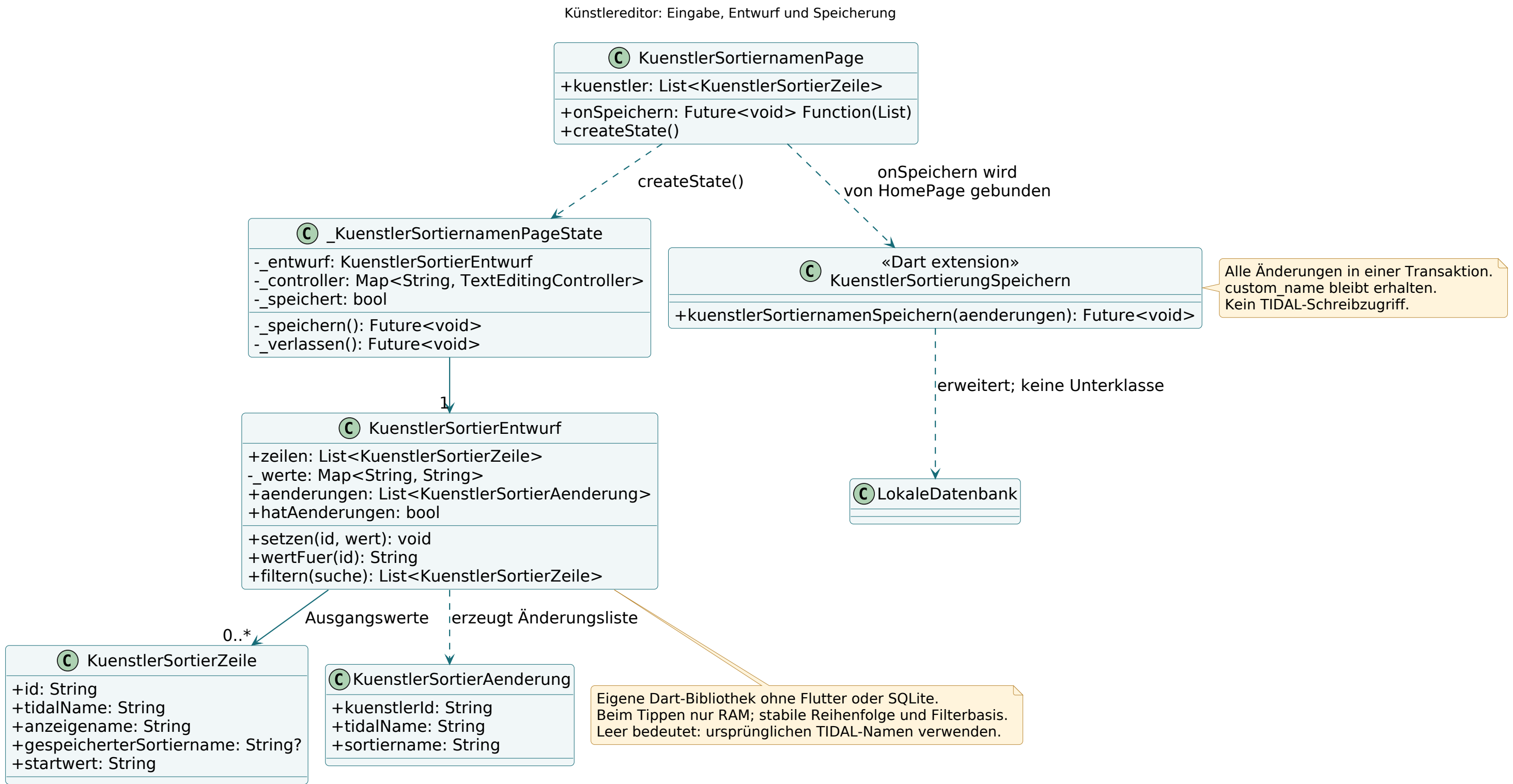


Bearbeitbare Quelle: [PlantUML/03_Klassen_Sitzung.puml](#) · Vektorausgabe: [SVG/03_Klassen_Sitzung.svg](#)

Modell 04 · Künstlereditor: Eingabe, Entwurf und Speicherung

Der Entwurf trennt vorläufige Eingaben vom gespeicherten Stand. Eine Dart-Extension ist keine Vererbung und erhält deshalb keinen Generalisierungspfeil.

Quellstellen unter lib/, sofern nicht anders angegeben: models/kuenstler_sortierung.dart; pages/kuenstler_sortiernamen_page.dart; data/kuenstler_sortierung_speichern.dart

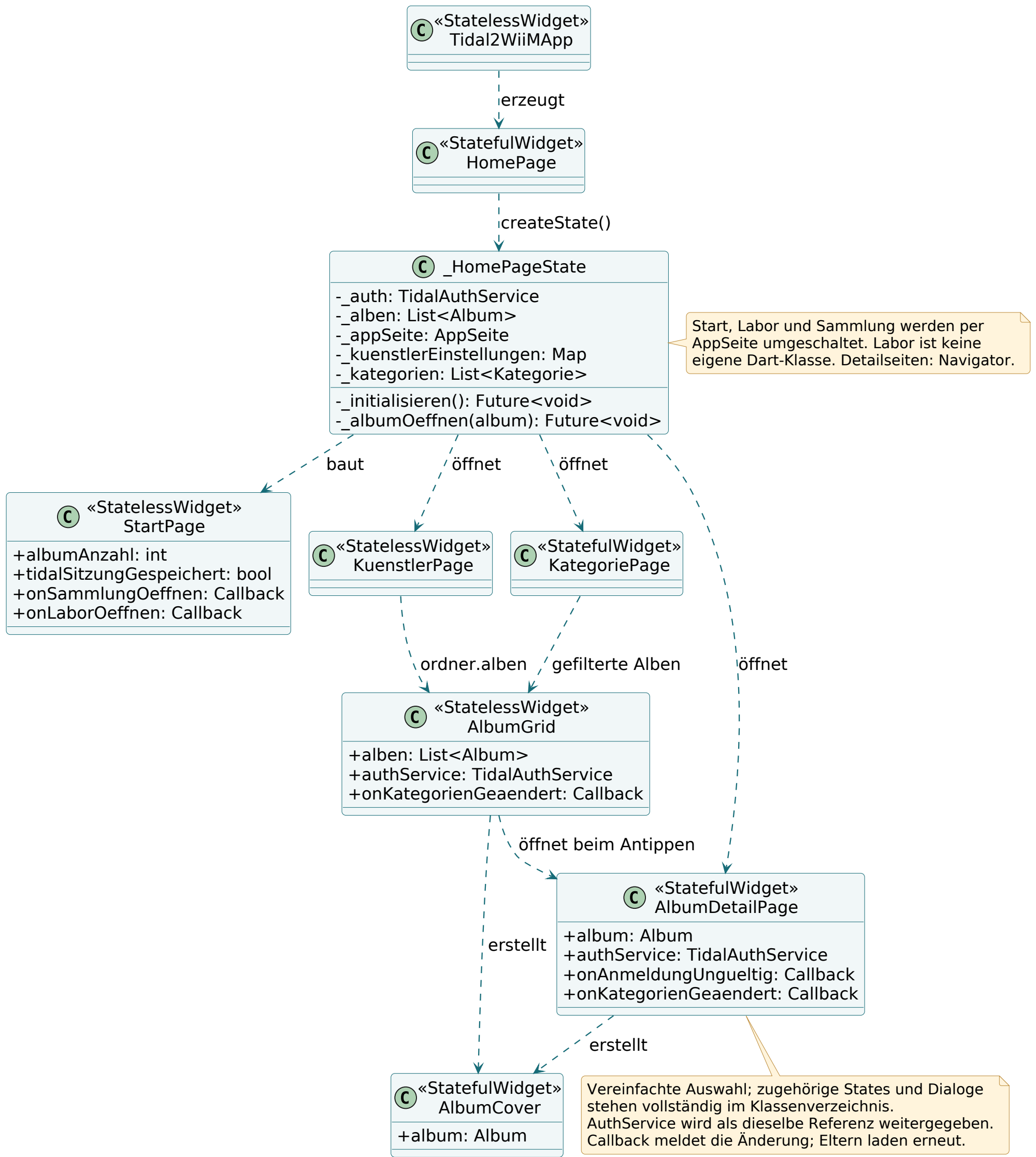


Modell 05 · Oberfläche: Objekte und Rückmeldungen

Widgets erhalten Daten und Funktionen über Konstruktoren. Ein Callback kann nach dem Speichern das Neuladen der übergeordneten Ansicht auslösen.

Quellstellen unter lib/, sofern nicht anders angegeben: main.dart; pages/*; widgets/album_grid.dart; widgets/album_cover.dart

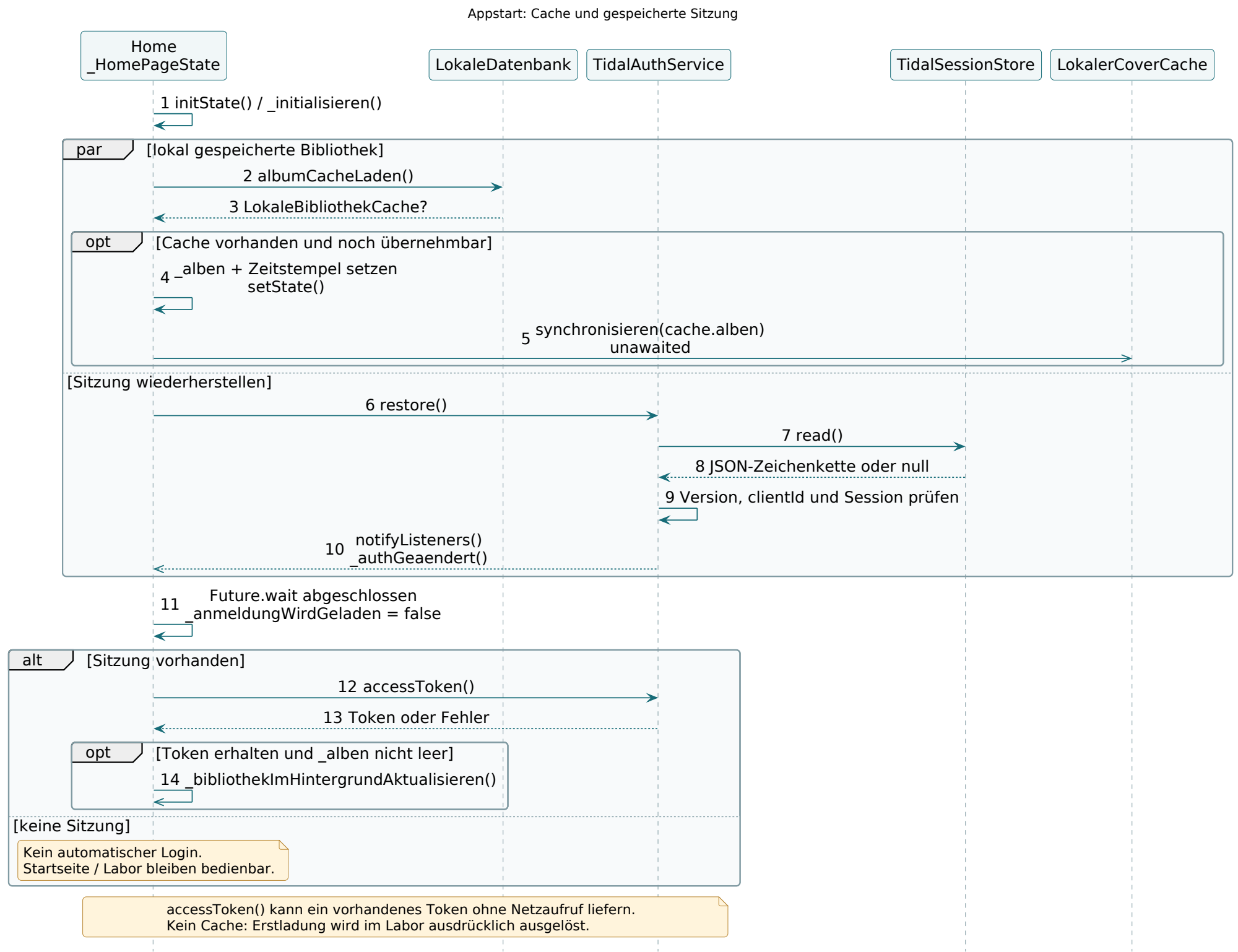
Oberfläche: Objekte und Rückmeldungen



Modell 06 · Appstart: Cache und gespeicherte Sitzung

Zwei asynchrone Aufgaben starten gemeinsam. Ein geladener Cache erscheint unabhängig vom Abschluss einer späteren Netzwerkaktualisierung; Tokens und Alben werden getrennt gelesen.

Quellstellen unter lib/, sofern nicht anders angegeben: pages/home_page.dart: _initialisieren(), _bibliothekAusCacheLaden(); services/tidal_auth_service.dart: restore()



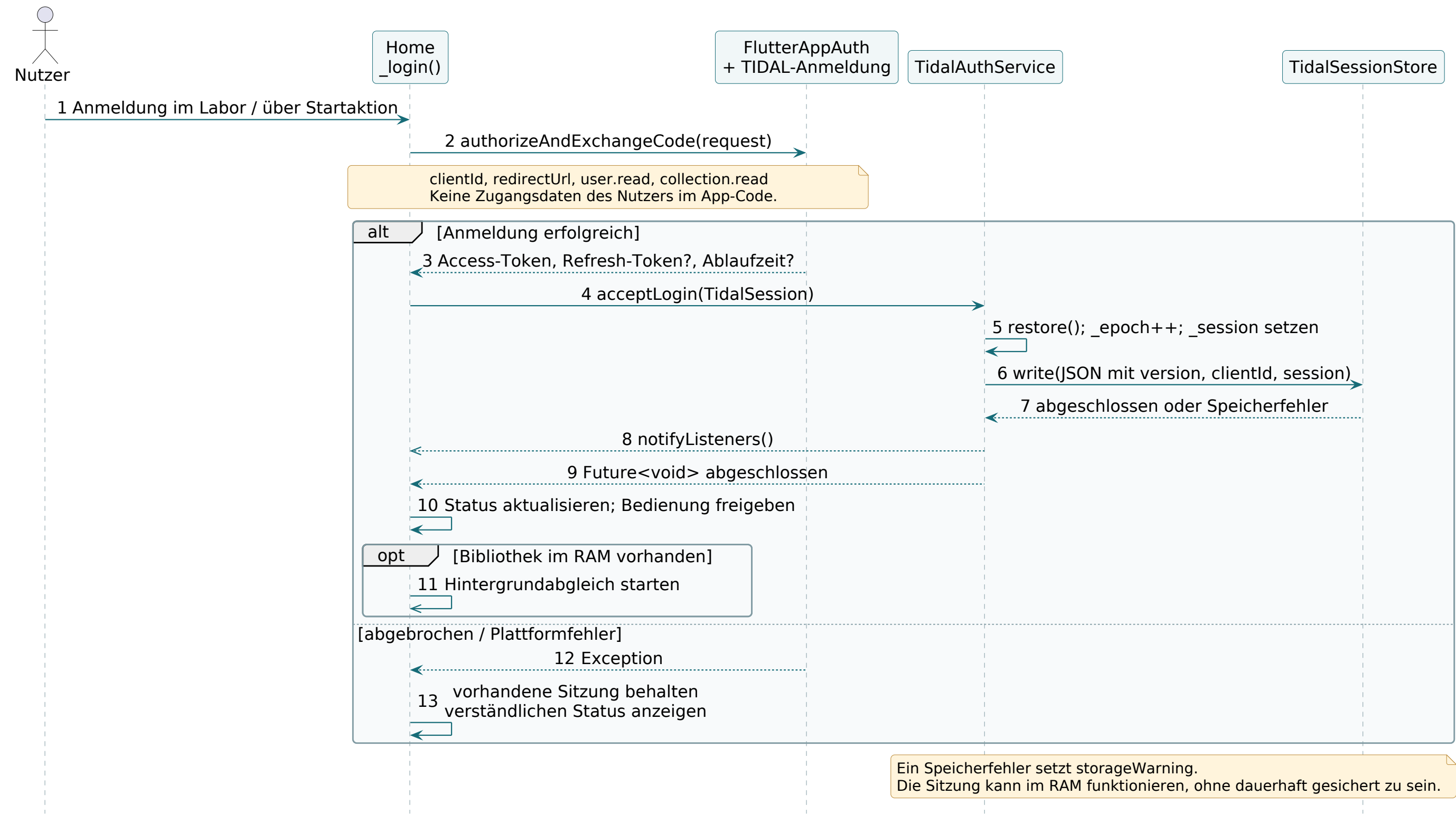
Bearbeitbare Quelle: PlantUML/06_Sequenz_Appstart.puml · Vektorausgabe: SVG/06_Sequenz_Appstart.svg

Modell 07 · Interaktive Anmeldung und sicheres Speichern

Erfolgreiche Anmeldung und erfolgreiche dauerhafte Speicherung sind zwei verschiedene Ergebnisse. Diese Unterscheidung verhindert falsche Versprechen beim nächsten Appstart.

Quellstellen unter lib/, sofern nicht anders angegeben: pages/home_page.dart: _login(); services/tidal_auth_service.dart: acceptLogin(), _persist()

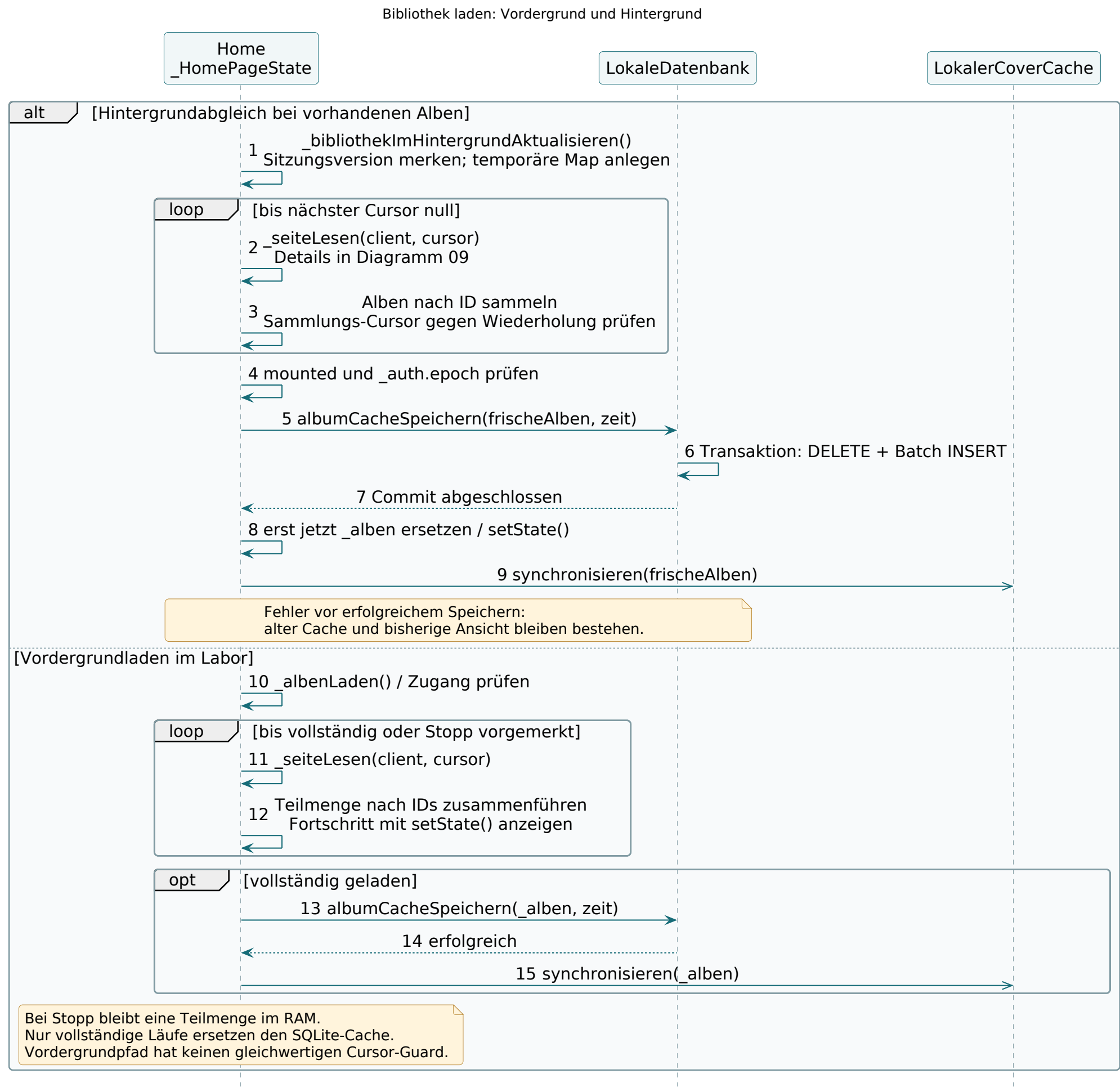
Interaktive Anmeldung und sicheres Speichern



Modell 08 · Bibliothek laden: Vordergrund und Hintergrund

Beim Hintergrundabgleich bleibt die alte Ansicht bis zum vollständigen neuen Stand erhalten. Das Vordergrundladen zeigt dagegen bereits gelesene Teilmengen.

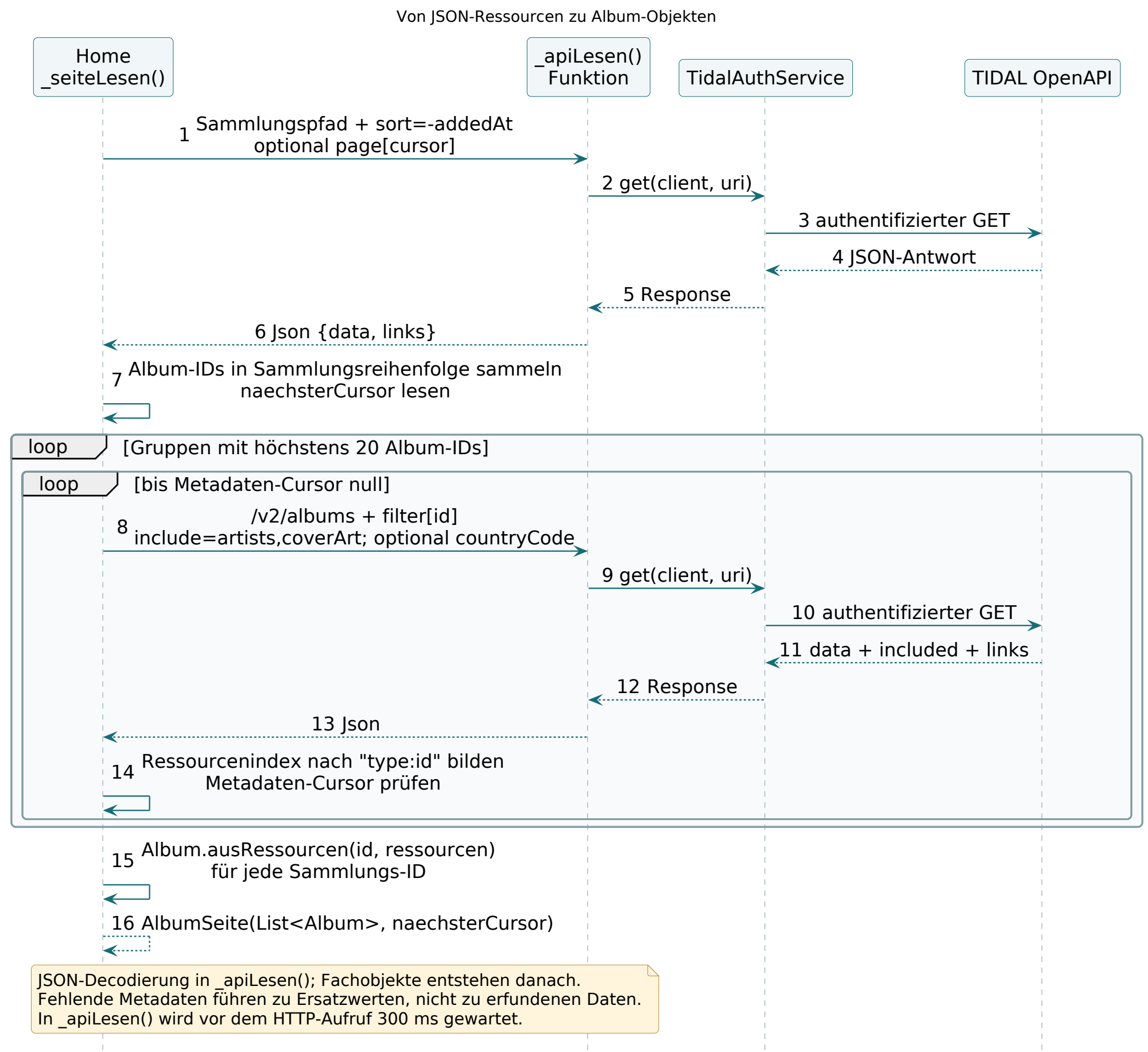
Quellstellen unter lib/, sofern nicht anders angegeben: pages/home_page.dart: _albenLaden(), _bibliothekImHintergrundAktualisieren(); data/lokale_datenbank.dart: albumCacheSpeichern()



Modell 09 · Von JSON-Ressourcen zu Album-Objekten

Sammlungs-IDs und Album-Metadaten kommen aus getrennten Abfragen. Der Ressourcenindex löst die Beziehungen auf Künstler und Cover auf.

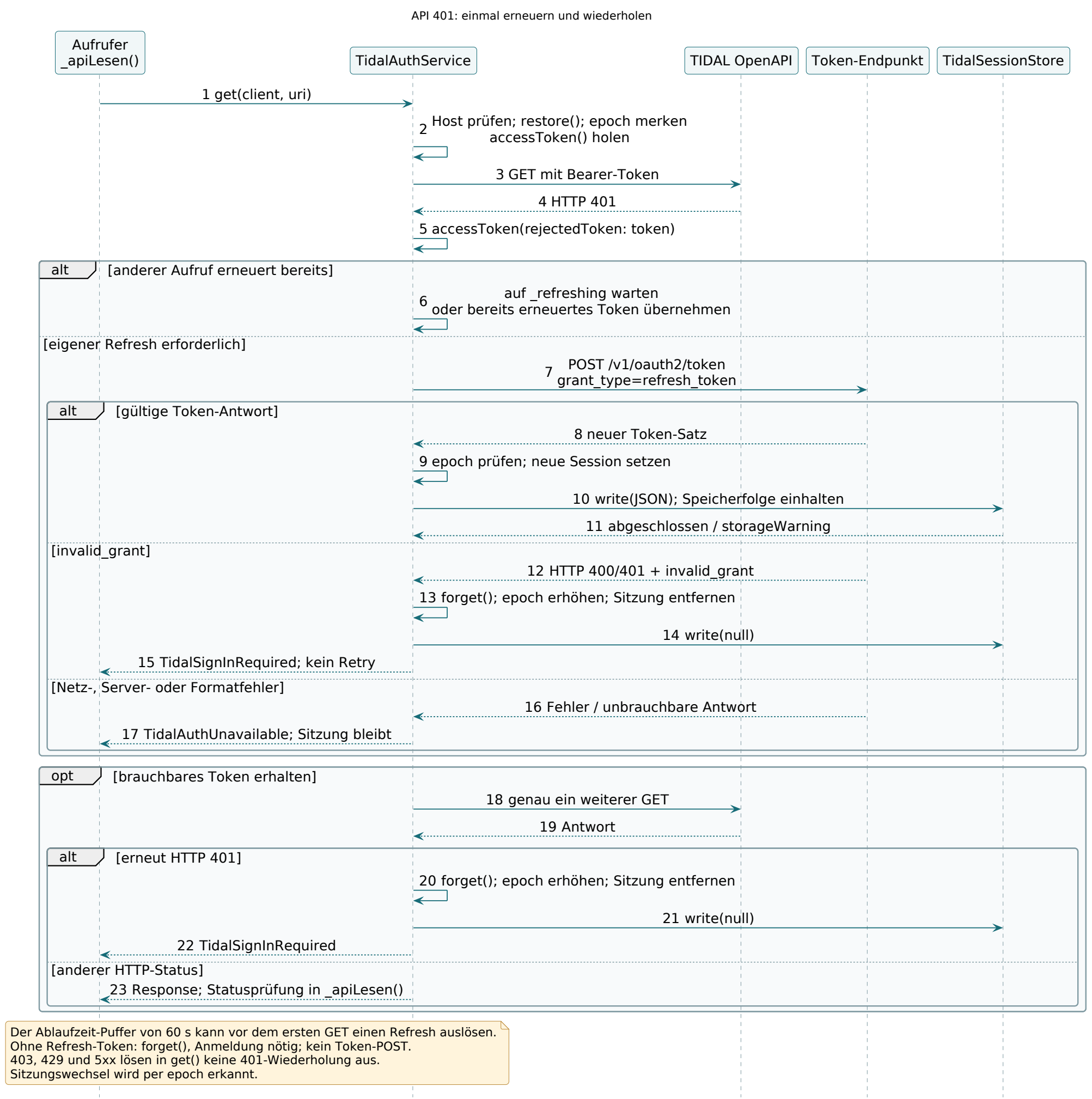
Quellstellen unter lib/, sofern nicht anders angegeben: pages/home_page.dart: _seiteLesen(); services/tidal_api.dart; models/modelle.dart: Album.ausRessourcen()



Modell 10 · API 401: einmal erneuern und wiederholen

Ein Netzwerkfehler ist kein Beweis für eine ungültige Anmeldung. Parallele Aufrufe teilen den Refresh; eine zweite Ablehnung endet ohne Endlosschleife.

Quellstellen unter lib/, sofern nicht anders angegeben: services/tidal_auth_service.dart: get(), accessToken(), _refresh(), _persist()

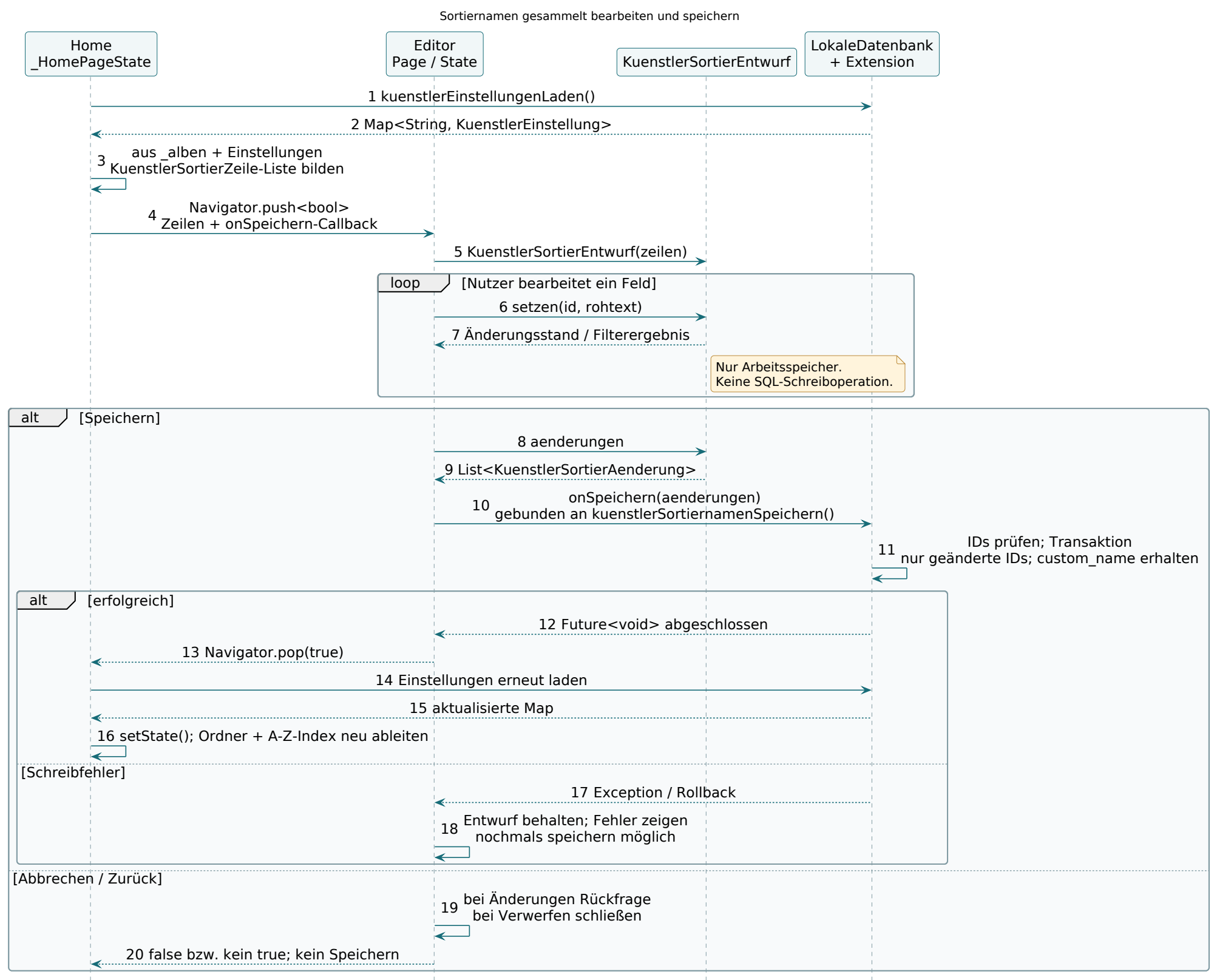


Bearbeitbare Quelle: PlantUML/10_Sequenz_Token401.puml · Vektorausgabe: SVG/10_Sequenz_Token401.svg

Modell 11 · Sortiernamen gesammelt bearbeiten und speichern

Kein Speichern pro Tastendruck: Eine bestätigte Änderungsliste wird gemeinsam geschrieben. Die Seite kennt das konkrete SQL nicht, sondern nur ihren Speicher-Callback.

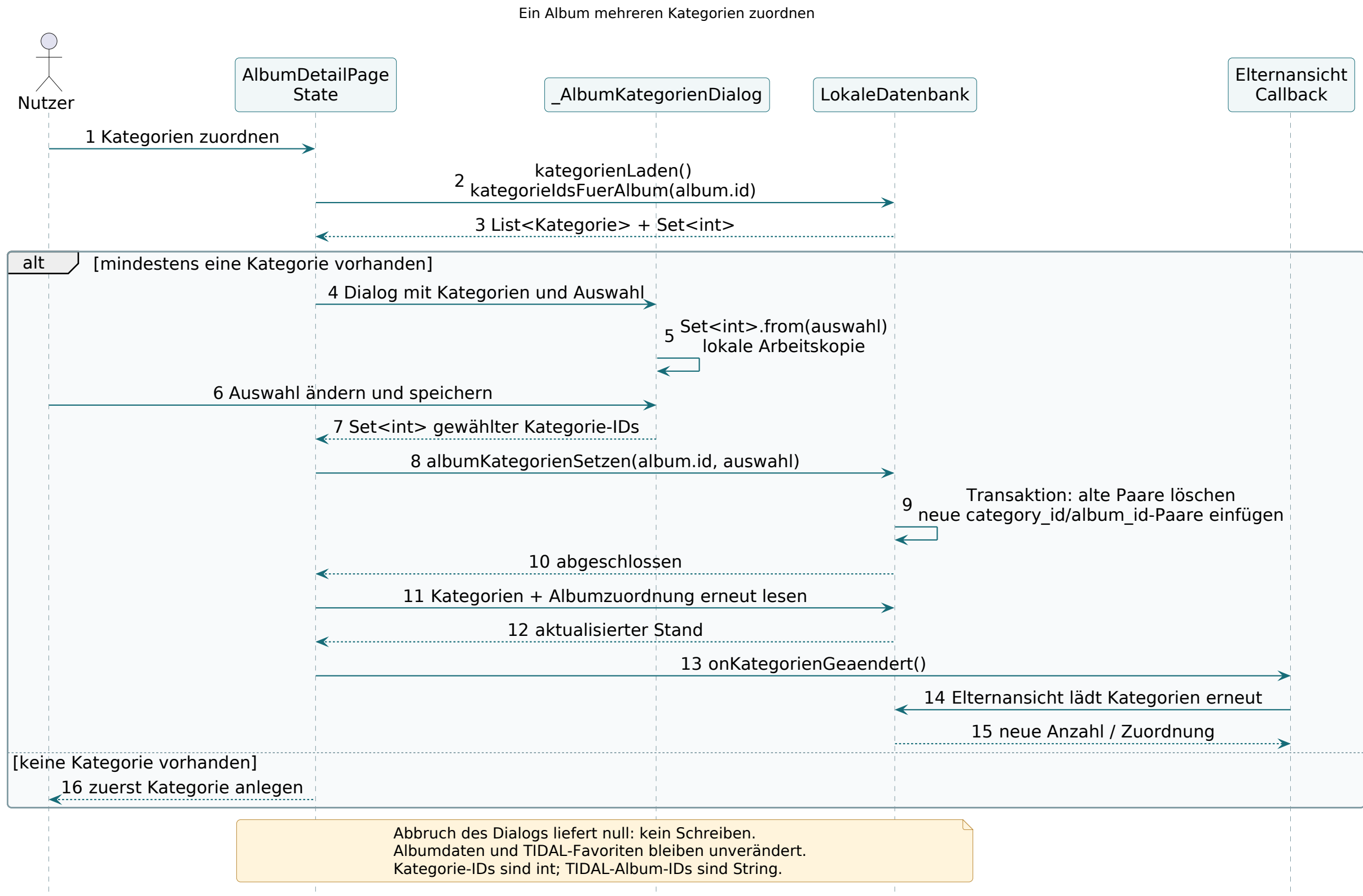
Quellstellen unter lib/, sofern nicht anders angegeben: pages/home_page.dart: _kuenstlerSortiernamenOeffnen(); pages/kuenstler_sortiernamen_page.dart; models/kuenstler_sortierung.dart; data/kuenstler_sortierung_speichern.dart



Modell 12 · Ein Album mehreren Kategorien zuordnen

Die Checkboxen liefern nur IDs. Erst die Detailseite veranlasst die Transaktion und meldet die Änderung an übergeordnete Seiten.

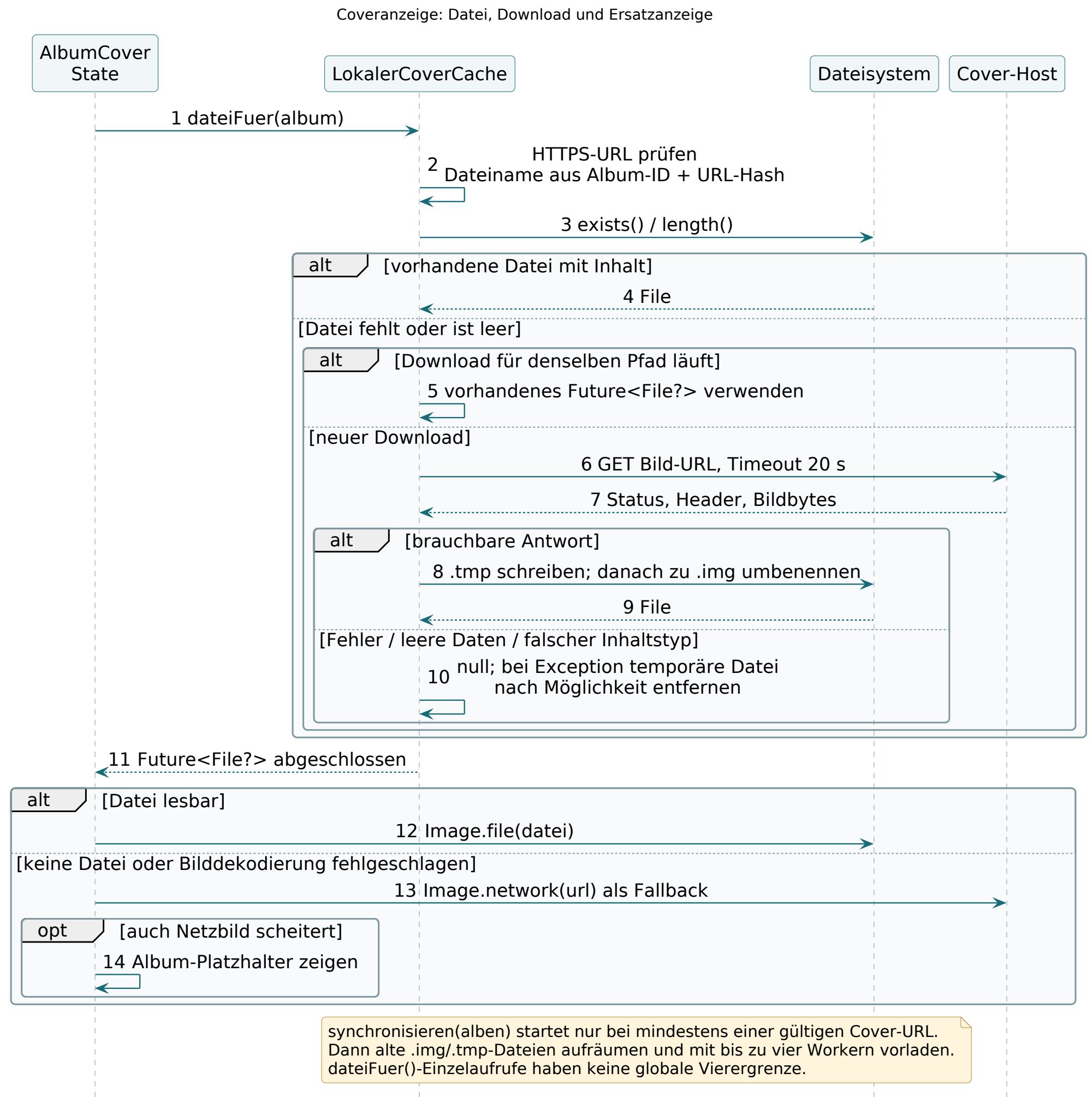
Quellstellen unter lib/, sofern nicht anders angegeben: pages/album_detail_page.dart: _kategorienBearbeiten(); widgets/dialoge.dart; data/lokale_datenbank.dart: albumKategorienSetzen()



Modell 13 · Coveranzeige: Datei, Download und Ersatzanzeige

Ein File ist kein Albumdatensatz. Die Oberfläche erhält eine Datei oder null; der Coverabruf verwendet keinen OAuth-Bearer-Header.

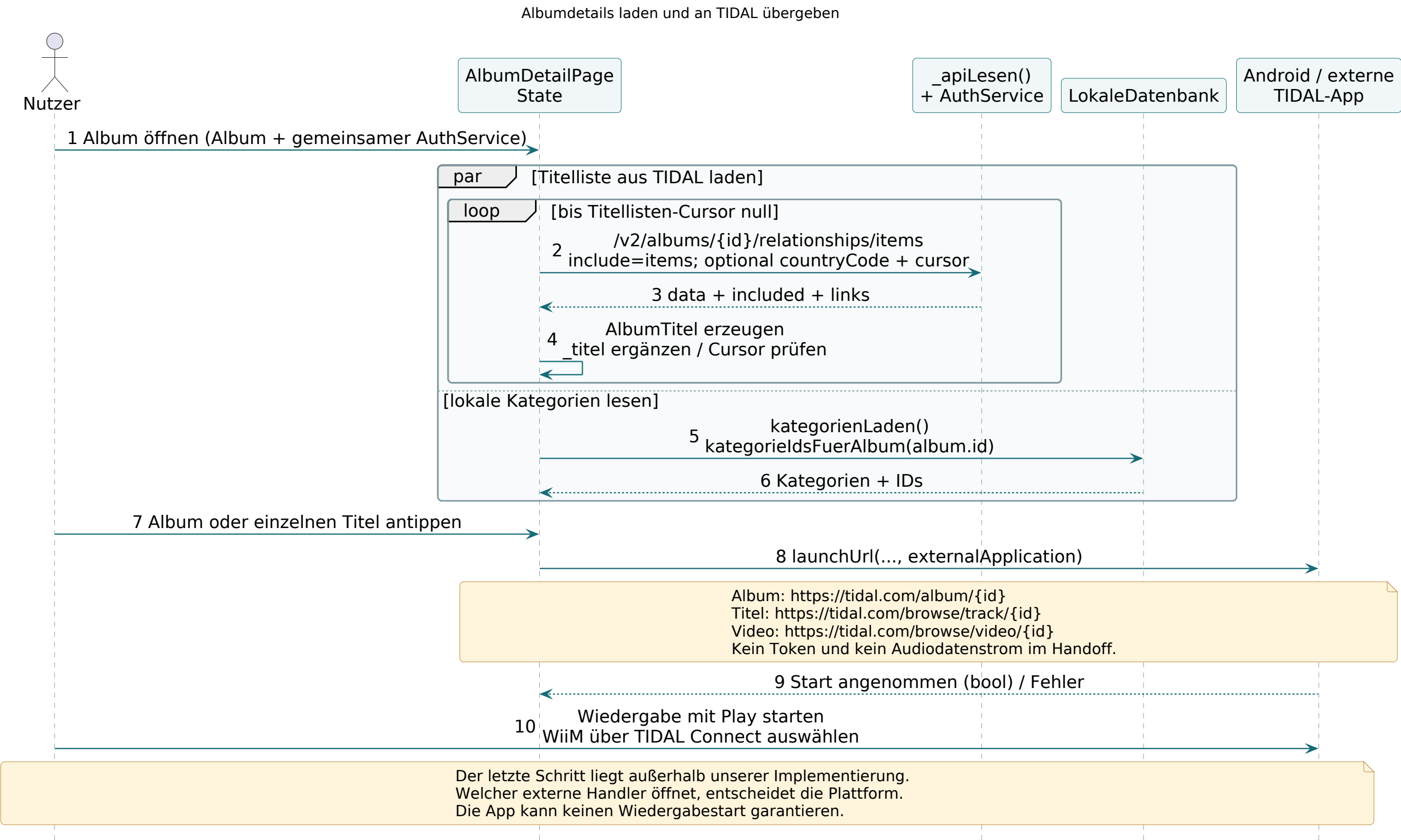
Quellstellen unter lib/, sofern nicht anders angegeben: services/cover_cache.dart; widgets/album_cover.dart



Modell 14 · Albumdetails laden und an TIDAL übergeben

Titelliste und Kategorien stammen aus verschiedenen Quellen. Ein externer Link ist eine Übergabe an eine andere App und kein eigener Musikplayer.

Quellstellen unter lib/, sofern nicht anders angegeben: pages/album_detail_page.dart: initState(), _titelLaden(), _inTidalOeffnen(), _titelInTidalOeffnen()



Fremdpakete und Tests

Direkte externe Pakete

Die angegebenen Paketversionen stammen aus dem pubspec.lock des dokumentierten Quellstands.

Paket	Lockfile-Version	Aufgabe
flutter	SDK	Widgets, MaterialApp, Navigator, setState, ChangeNotifier und Lebenszyklus.
flutter_appauth	12.1.0	Interaktive TIDAL-Anmeldung; liefert den ersten Token-Satz an HomePage.
http	1.6.0	OpenAPI-GET, Token-POST und Coverdownload; austauschbare Clients ermöglichen Tests.
sqlite	2.4.4	SQLite-Abfragen, Migrationen und Transaktionen auf dem Gerät.
path	1.9.1	Plattformgerechtes Verbinden von Datenbank- und Coverpfaden.
flutter_secure_storage	11.2.0	Sicherer Plattform-Speicher für das JSON-Tokenpaket.
url_launcher	6.3.2	Album-, Track- und Videolinks an eine externe Anwendung übergeben.
cupertino_icons	1.0.9	Im Projekt deklariert; im untersuchten lib-Code kein CupertinoIcons-Aufruf.
flutter_test	SDK	Unit-/Widget-Tests, WidgetTester und Flutter-Testumgebung.
flutter_lints	6.0.0	Statische Stil-/Analyse-Regeln über analysis_options.yaml; kein Laufzeitdienst.
flutter_launcher_icons	0.14.4	Entwicklungswerkzeug zum Erzeugen der App-Icons; kein Laufzeitdienst.

Die Dart-SDK-Bibliotheken dart:async, dart:convert und dart:io liefern asynchrone Abläufe, JSON/UTF-8 sowie Dateizugriffe. Flutter stellt unter anderem material.dart, services.dart und foundation.dart bereit. Diese Imports sind keine weiteren selbst entwickelten Projektpakete.

Konfiguration und Plattformordner

pubspec.yaml beschreibt Paket, Abhängigkeiten und Assets. pubspec.lock hält aufgelöste Versionen fest. analysis_options.yaml konfiguriert die statische Analyse. assets/ enthält Startbild und Icon. docs/datenmodell/ enthält die bereits vorhandenen Planungsmodelle.

android/ enthält die native Android-Einbettung, Gradle-Konfiguration, Manifest, OAuth-Rücksprung und Ressourcen. ios/, macos/, linux/ und web/ sind zusätzliche Plattformgerüste. Ihre Existenz belegt keine getestete Unterstützung dieser Plattformen. Das erklärte Zielgerät bleibt Android.

Vorhandene Tests

Datei	Schwerpunkt	Was sie prüfen / bereitstellen
test/kuenstler_sortierung_test.dart	Entwurfslogik	Nur geänderte IDs, Leerwert-Rückfall, stabile Filter/Reihenfolge, getrennte IDs trotz gleicher Namen und ungültige Eingaben.
test/kuenstler_sortiernamen_page_test.dart	Editor-Oberfläche	Speichern/Abbrechen, Android-Zurück, Speicherfehler, erhaltene Eingaben, Tastaturfokus und verschiedene Bildschirm-/Textgrößen.
test/tidal_auth_service_test.dart	Sitzungsdienst	Wiederherstellung, Ablaufzeit, gemeinsamer Refresh, Netz-/Serverfehler, invalid_grant, Speicherwarnung und genau ein 401-Retry.
test/widget_test.dart	Start/Labor	Startaktionen und Trefferflächen, dynamische Statuswerte, Wiederherstellung/Anmeldung vergessen und Layoutvarianten.
test/support/memory_session_store.dart	Test-Doppel	MemorySessionStore speichert nur im RAM, zählt Schreibvorgänge und simuliert Schreibfehler; kein eigenständiger Testfall.

Die Übersicht beschreibt die vorhandenen Testquellen. Ergebnisse einer erneuten Test- oder Build-Ausführung sind nicht Bestandteil dieser Dokumentation. Widget-Tests mit injizierten Daten oder Mock-Clients prüfen isolierte Abläufe; eine reale API-, SQLite- oder Tablet-Integration erfordert zusätzliche Integrationstests.

Für SQLite-Transaktionen und den Covercache liegt im untersuchten test/-Ordner kein eigener direkter Integrationstest vor. Ein kompletter Ende-zu-Ende-Nachweis mit TIDAL und WiiM wäre eine zusätzliche Prüfung.

Vollständiges Klassenverzeichnis

46 Klassen-/Interface-Deklarationen in lib/. Die Modulkennung verweist auf den Steckbrief mit dem vollständigen Dateipfad. Framework-Klassen und MemorySessionStore aus den Tests sind hier nicht mitgezählt.

Name	Modul	Aufgabe
LokaleDatenbank	M06	SQLite-Zugriffe und Schema 3.
Tidal2WiiMApp	M01	Wurzelwidget, Theme und HomePage.
KuenstlerSortierZeile	M05	Unveränderte Ausgangsdaten einer Editorzeile.
KuenstlerSortierAenderung	M05	Datenträger für eine bestätigte Sortieränderung.
KuenstlerSortierEntwurf	M05	Arbeitsspeicher für Eingaben, Filter und Änderungserkennung.
KuenstlerEinstellung	M04	Eigener Anzeigename und lokaler Sortiername.
Kategorie	M04	Kategorie-ID, Name und Menge von Album-IDs.
Kuenstler	M04	TIDAL-Künstler-ID und Name.
Album	M04	Album-Metadaten und beteiligte Künstler.
LokaleBibliothekCache	M04	Geladene Albumliste mit Speicherzeit.
AlbumTitel	M04	Ein Track oder Video im Zustand der Detailseite.
KuenstlerOrdner	M04	Aus Alben abgeleitete Künstleransicht.
AlbumSeite	M04	Ergebnis einer API-Sammlungsseite mit Folgekursor.
AlbumDetailPage	M13	Konfiguration für ein ausgewähltes Album.
_AlbumDetailPageState	M13	Titelliste, Kategorien und externe Übergabe.
HomePage	M11	Konfiguration der Hauptseite und optionale Test-Abhängigkeiten.
_HomePageState	M11	Zentrale Koordination von Zustand, Datenladen und Navigation.
KategoriePage	M15	Konfiguration einer Kategorieansicht.
_KategoriePageState	M15	Kategorie lesen, aktuelle Albumobjekte filtern, Anzeige erneuern.
KuenstlerPage	M14	Künstlerüberschrift und gemeinsames Albumraster.
KuenstlerSortiernamenPage	M16	Konfiguration des Editors und Speicher-Callback.
_KuenstlerSortiernamenPageState	M16	Entwurf, Eingabefelder, Fokus und Abbruch-/Speicherablauf.
StartPage	M12	Startmotiv und vom Elternwidget gelieferte Statusdaten.
_StartAktion	M12	Beschriftete, tatsächlich antippbare Trefferfläche.
_StartStatusLeiste	M12	Album-/TIDAL-Status und Aktualisierungsaktion.

Name	Modul	Aufgabe
_StartTidalZeichen	M12	Zeichnet das Statussymbol als CustomPainter.
_StartBibliothekZeichen	M12	Zeichnet das Bibliothekssymbol als CustomPainter.
LokalerCoverCache	M10	Dateisuche, Download, Vorladen und Bereinigung der Cover.
ApiFehler	M09	HTTP-Status und API-Pfad als Fehlerobjekt.
TidalSessionStore	M08	Interface zum Lesen/Schreiben des Session-Pakets.
SecureTidalSessionStore	M08	Produktiver Store auf Basis von flutter_secure_storage.
TidalSession	M08	Token-Satz und Ablaufzeit.
TidalSignInRequired	M08	Eine neue Anmeldung ist erforderlich.
TidalAuthUnavailable	M08	Vorübergehender Zugangsfehler ohne automatisches Verwerfen der Sitzung.
TidalSessionChanged	M08	Ein laufender Vorgang gehört nicht mehr zur aktuellen Sitzung.
TidalAuthService	M08	Gemeinsame Sitzung, Token-Erneuerung, gesicherte API-GETs.
AlbumCover	M18	Konfiguration der Coveranzeige für ein Album.
_AlbumCoverState	M18	Datei-Future, Ladeanzeige und Bild-Fallback.
AlbumGrid	M17	Gemeinsames Raster für Künstler-/Kategorie-Unterseiten.
_KategorieNameDialog	M19	Konfiguration des Kategorienamen-Dialogs.
_KategorieNameDialogState	M19	Textcontroller, Pflichtfeldprüfung und Namensrückgabe.
_KuenstlerDialogErgebnis	M19	Einstellung oder ausdrücklicher Zurücksetzen-Wunsch.
_KuenstlerBearbeitenDialog	M19	Konfiguration der einzelnen Künstleranpassung.
_KuenstlerBearbeitenDialogState	M19	Controller für Anzeige-/Sortiername und Ergebnissrückgabe.
_AlbumKategorienDialog	M19	Konfiguration der Kategorieauswahl.
_AlbumKategorienDialogState	M19	Bearbeitet eine Kopie der ausgewählten IDs.

Zusätzlich: die Dart-Extension KuenstlerSortierungSpeichern (M07), die Enums Sortierung, SammlungAnsicht und AppSeite (M03) sowie das Typalias Json (M02). Die Funktionen in core/ und tidal_api.dart sind keine weiteren Klassen.

Mit den Quellen weiterarbeiten

Das ZIP ist ein eigenständiges Dokumentationspaket. Es enthält die Lesefassung, Markdown-Kapitel, 19 einzeln nutzbare Modulsteckbriefe und die 14 Modelle als PlantUML, SVG und PNG. Die App-Quellen werden nicht ein zweites Mal mitgeliefert; die Steckbriefe verweisen auf den festgehaltenen GitHub-Stand.

Modelle rendern

Java und eine vorhandene plantuml.jar genügen. Die mitgelieferten Skripte schreiben die Ausgaben nach SVG/ und PNG/.

```
.\Modelle_rendern.ps1 -PlantUmlJar "C:\Tools\plantuml\plantuml.jar"
```

Oder mit Python und Java:

```
python Modelle_rendern.py /pfad/zu/plantuml.jar
```

Diese Fassung wurde mit PlantUML 1.2025.10 lokal gerendert. Für Komponenten- und Klassenmodelle wird Smetana verwendet; externes Graphviz ist dafür nicht nötig. Es wurde kein Projektcode an einen öffentlichen Renderdienst übertragen.

Eine Änderung nachvollziehbar dokumentieren

1. Den betroffenen Modulsteckbrief und das zugehörige Modell lesen.
2. Zuerst klären, welche Daten sich ändern und wo sie gespeichert werden.
3. Code und PlantUML-Quelle gemeinsam anpassen; anschließend neu rendern.
4. Aussagen durch geeignete Prüfungen absichern und den Quellstand nennen.
5. Verwendete Hilfsmittel einschließlich ChatGPT ausdrücklich angeben.